모의해킹 결과보고서

작성일: 2026년 04월 06일

버전: 1.0

[목차]

1. 개요 6
1.1.수행 목적 6
1.2.수행 대상 6
1.3.수행 장소 6
1.4.수행 일정 7
1.5.수행 인력 7
2. 세부 수행 내용 8
2.1. 웹 취약점 진단 8
3. 수행 결과 12
3.1. 웹 서비스 취약점 점검 결과 12
3.1.1.총평 12
3.1.2.결과 요약 13
4. 취약점 진단 상세 내용 14
4.1. Vuln-Camp (127.0.0.1) 14
4.1.1.SQL Injection 14
4.1.2.SSRF 17
4.1.3.파일 업로드 19
4.1.4.파일 다운로드 21
4.1.5.프로세스 검증 누락 23
4.1.6.불충분한 인가 28
4.1.7.XSS 30
4.1.8.CSRF 35
4.1.9. 정보 누출 37
4.1.10. 서버 정보 노출 38

5. 웹 취약점 보안대책 39
5.1. SQL 인젝션 39
5.2. SSRF 39
5.3. 파일 업로드 40
5.4. 파일 다운로드 40
5.5. 프로세스 검증 누락 41
5.6. 불충분한 인가 41
5.7. XSS 42
5.8. CSRF 42
5.9. 정보 누출 43
5.10. 서버 정보 노출 43

[표 목차]

[표 1] 수행 범위 및 대상 6

[표 2] 수행 장소 6

[표 3] 수행 일정 7

[표 4] 수행 인력 7

[표 5] 웹 취약점 진단 점검 항목 11

[표 6] 결과 요약 13

[그림 목차]

[그림 1] QnA 게시판 날짜 검색 기능 확인 14
[그림 2] QnA 게시판 날짜 검색 요청 파라미터 확인 14
[그림 3] 참 조건 SQL Injection 구문 삽입 결과 15
[그림 4] 거짓 조건 SQL Injection 구문 삽입 결과 15
[그림 5] 소스코드 노출을 통한 SSRF 컨트롤러 확인 16
[그림 6] SSRF를 통한 내부 API 응답 정보 확인 16
[그림 7] QnA 게시글 첨부파일 업로드 기능 확인 17
[그림 8] JSP 파일 업로드 요청 17
[그림 9] JSP 파일 업로드 성공 확인 18
[그림 10] JSP 파일 업로드 후 서버 측 실행 확인 19
[그림 11] filename 파라미터 경로 조작 20
[그림 12] 경로 조작을 통한 서버 내부 설정 파일 다운로드 확인 21
[그림 13] 상품 상세 페이지 정상 금액 확인 21
[그림 14] 주문서 작성 페이지 정상 금액 확인 22
[그림 15] 결제 페이지 정상 금액 확인 22
[그림 16] 카드 결제 모달 정상 금액 확인 23
[그림 17] 결제 금액 파라미터 변조 요청 23
[그림 18] 변조된 결제 금액 주문 생성 응답 확인 24
[그림 19] 변조된 결제 금액 주문 내역 반영 확인 24
[그림 20] 클라이언트 측 쿠폰 검증 로직 노출 확인 25
[그림 21] 쿠폰 생성 규칙 분석 및 유효 쿠폰 코드 생성 25
[그림 22] 유효 쿠폰 코드 적용 확인 26
[그림 23] 쿠폰 적용 후 할인 금액 반영 확인 26
[그림 24] 일반 사용자 상태 공지사항 화면 확인 27
[그림 25] 쿠키 변조를 통한 관리자 기능 노출 확인 27
[그림 26] 관리자 공지사항 작성 요청 28
[그림 27] 쿠키 변조 후 공지사항 등록 결과 확인 29
[그림 28] QnA 게시글 작성 기능 확인 30
[그림 29] XSS 구문 삽입 및 저장 확인 30
[그림 30] <script> 구문 필터링 확인 31
[그림 31] 이벤트 핸들러 기반 XSS 구문 저장 확인 32
[그림 32] Stored XSS 실행 확인 33
[그림 33] 비밀번호 변경 기능 확인 34
[그림 34] CSRF를 통한 비밀번호 변경 요청 성공 확인 35
[그림 35] CSRF 공격 페이지 실행 결과 확인 36
[그림 36] HTML 주석 내 관리자 페이지 경로 노출 확인 37
[그림 37] 디버그 API를 통한 서버 소스코드 노출 확인 38

1. 개요

1.1. 수행 목적

본 웹 진단 프로젝트는 Vuln-Camp 웹 애플리케이션을 대상으로 공격자의 관점에서 웹 서비스에 존재하는 취약점 악용 및 중요 정보 탈취를 목표로 수행하였다. 이를 통해 운영 중인 웹 서비스에 노출된 보안 위협 요소와 취약점을 사전에 식별하고, 전반적인 보안 수준을 높이는 것을 목적으로 한다.

1.2. 수행 대상

본 점검은 로컬 환경에서 구동되는 취약점 실습용 웹 애플리케이션 Vuln-Camp를 단일 대상으로 수행하였다.

번호	대상명	도메인	IP
1	Vuln-Camp	127.0.0.1:3000	127.0.0.1

[표 1] 수행 범위 및 대상

1.3. 수행 장소

수행 장소 및 네트워크 IP 정보는 다음과 같으며, 물리적 보안이 적용된 환경에서 수행하였음.

장소	IP	비고
내부 실습 환경	내부 IP	실습 환경 범위내에서 진행하였음

[표 2] 수행 장소

1.4. 수행 일정

본 프로젝트는 2026.04.04 ~ 2026.04.06까지 3일간 수행하였으며, 세부 일정은 다음과 같음.

구분	내용	일정
사전 현황 파악	일정 협의 및 대상 확인	2026 년 04 월 04 일
	환경설정 및 점검 대상 분석	2026 년 04 월 04 일
웹 취약점 진단 및 보안성 강화 업무	대외 서비스 웹 취약점 진단	2026 년 04 월 04 일 ~ 2026 년 04 월 6일
보고서	결과 정리 및 보고서 작성	2026 년 04 월 05 일 ~ 2026 년 04 월 6일

[표 3] 수행 일정

1.5. 수행 인력

소속	이름	수행 내용
-	유진욱	- 대외 서비스 웹 모의해킹 및 보고서 작성

[표 4] 수행 인력

2. 세부 수행 내용

2.1. 웹 취약점 진단

웹 취약점 진단 항목은 금융위원회『전자금융감독규정』, 금융감독원『금융회사 정보기술(IT)부문 보호업무 모범규준』, 행정안전부『주요정보통신기반시설 취약점 분석평가 기준』, 국제 웹 보안 표준기구 OWASP TOP 10 을 기반으로 34 개 점검 항목으로 웹 취약점 진단을 수행함

분류	취약점 점검 항목			
계정 취약점	취약점	**[IW-01] 약한 문자열 강도**	위험도	중
	패스워드 설정 시 영문/숫자/특수문자 3 가지 이상 조합일 경우에는 8 자리, 2 가지 이상 조합일 경우 10 자리 이상으로 패스워드를 설정하도록 강제하지 않을 경우 취약. 기존에 생성된 계정 중에서 추측 가능한 패스워드를 사용할 경우 취약			
	취약점	**[IW-02] 인증 실패 횟수 제한 미존재**	위험도	중
	인증 실패 횟수 제한 미존재로 무차별 공격에 의해 계정 탈취 위험 존재 (로그인, 2 Factor, 추가 인증 등)			
인젝션	취약점	**[IW-03] SQL Injection**	위험도	상
	파라미터 변조를 통해 DB 내 저장된 데이터 조회 또는 권한 획득 가능 시 취약 (공격 구문 필터링으로 DB 조회는 불가능하나 공격 허용할 경우 포함)			
	취약점	**[IW-04] 운영체제 명령 실행**	위험도	상
	파라미터 변조를 통해 웹 서버 OS 의 명령 실행이 가능 시 취약			
	취약점	**[IW-05] XPATH Injection**	위험도	상
	파라미터 변조를 통해 저장된 데이터 조회 또는 권한 획득 가능 시 취약			
	취약점	**[IW-06] LDAP Injection**	위험도	상
	파라미터 변조를 통해 저장된 데이터 조회 또는 권한 획득 가능 시 취약			
	취약점	**[IW-07] SSI Injection**	위험도	상
	파라미터 변조를 통해 악의적인 파일을 Include 할 수 있는 경우 취약			
	취약점	**[IW-08] XML 외부 개체 (XXE)**	위험도	상
	XML 문서에서 동적으로 외부 URI 및 리소스를 포함시킬 수 있는 External Entity 를 사용하여 서버의 로컬 파일을 열람하거나, 서비스 거부 공격이 가능할 경우 취약			

분류	취약점 점검 항목			
취약한 요청 및 응답 처리	취약점	**[IW-09] HTTP 응답 분할**	위험도	하
	HTTP 요청 시 전달된 파라미터가 HTTP 응답 헤더에 포함되어 다시 사용자에게 전달될 경우, 파라미터에 개행 문자(CR, LF)를 삽입하여 HTTP 요청 시 응답이 2 개로 분할되어 오는 경우 취약			
	취약점	**[IW-10] 신뢰되지 않은 URL 접속 유도**	위험도	하
	전달되는 파라미터 값에 URL 이 포함되어 해당 주소로 자동으로 이동되는 경우 취약			
	취약점	**[IW-11] Server-Side Request Forgery (SSRF)**	위험도	상
	파라미터 값, URL 변조를 통해 로컬 및 원격 리소스에 접근이 가능한 경우 취약			
부적절한 파일 처리	취약점	**[IW-12] 파일 업로드**	위험도	상
	파일 업로드 기능 존재 시 서버 사이드 스크립트 언어로 작성된 파일(ASP, PHP, JSP 등) 또는 허용되지 않은 악의적인 콘텐츠(실행 파일 등)를 업로드 및 실행 가능 시 취약 (PUT 메서드를 이용한 업로드 포함)			
	취약점	**[IW-13] 파일 다운로드**	위험도	상
	파일 다운로드 기능 존재 시 파라미터 값을 변조(Directory Traversal)하여 서버 내 리소스 및 중요파일(설정파일, 소스파일 등) 다운로드 가능 시 취약 (웹 루트 한정 시도 포함)			
인증 오류	취약점	**[IW-14] 프로세스 검증 누락**	위험도	중
	로그인하지 않은 세션에서 사용자 권한이 필요한 웹 페이지 접근 가능 시 취약			
	취약점	**[IW-15] 불충분한 인증**	위험도	중
	중요정보 페이지(개인정보 및 패스워드 변경 페이지 등) 접근 시 추가 인증을 하지 않는 경우 취약			
	취약점	**[IW-16] 불충분한 인가**	위험도	중
	파라미터 변조 및 서버로부터 전달된 응답 웹 페이지(HTML, JS, vbs 등)의 코드 변조를 통해 인증 우회 또는 권한 획득이 가능한 경우, 사용자 권한으로 로그인 한 상태에서 관리자 권한이 필요한 메뉴 또는 웹 페이지 접근이 가능하거나 타 사용자 정보 확인이 가능할 경우 취약			

분류	취약점 점검 항목			
악성 스크립트 허용	취약점	**[IW-17] Cross Site Scripting (XSS)**	위험도	중
	웹 페이지 내에 사용자 입력 값 또는 게시판과 같은 저장 기능에서 HTML 태그 및 Javascript 등의 스크립트 삽입이 가능하여 사용자의 세션을 탈취할 수 있거나, 사이트의 강제 이동, 악성 프로그램의 설치 등 악의적인 행위가 가능할 경우 취약 (Stored 방식, Reflected 방식 모두 포함)			
	취약점	**[IW-18] Cross Site Request Forgery (CSRF)**	위험도	상
	XSS 공격과 같이 사용자 입력 값 또는 게시판과 같은 저장 기능에서 사용자가 의도치 않은 상태에서 HTML 태그 및 Javascript 의 실행을 통한 행위(권한 추가/변경/삭제, 글 삭제/변경 등) 가능 시 취약			
보안 설정 오류	취약점	**[IW-19] 부적절한 메서드 허용**	위험도	하
	웹 서버에 GET, POST 메서드 외에 TRACE 메서드 등과 같은 불필요한 메서드 기능이 허용되어 있을 경우 취약 (필요에 의해 사용하는 경우 제외)			
	취약점	**[IW-20] 디렉토리 인덱싱**	위험도	하
	웹 서버에 경로 탐색 기능이 설정되어 웹 루트 하위의 디렉토리 및 파일 목록 조회가 가능할 경우 취약			
	취약점	**[IW-21] 불필요한 포트 오픈**	위험도	하
	웹 서비스 외에 불필요한 서비스 포트가 개방되어 있고 접근이 가능할 경우 취약			
	취약점	**[IW-22] 관리자 페이지 노출**	위험도	중
	웹 서비스 관리자 로그인 페이지, WAS 관리자 페이지 주소가 유추하기 쉬운 경우 취약			
	취약점	**[IW-23] 자동화 공격**	위험도	하
	웹 서비스에서 특정 기능의 반복적인 요청으로 금전적인 피해 가능성이 존재할 경우 취약			
정보 노출	취약점	**[IW-24] 정보 누출**	위험도	중
	소스, 응답 값 내 개인정보 또는 계정과 관련된 불필요한 정보가 노출된 경우, 로그인 페이지 내 계정의 존재 유·무 확인이 가능한 로그인 실패 메시지가 출력될 경우 취약			
	취약점	**[IW-25] 서버 정보 노출**	위험도	하
	서버 내 설치된 APP/OS 정보, OS 절대 경로 정보, DB/WAS 접속 정보 등의 서버 관련 중요 정보가 웹 페이지 상에 노출 시 취약 (웹 페이지 소스/주석 내 웹 소스, 패스워드, 시스템 정보,			

분류	취약점 점검 항목			
	개인정보, OS 종류, WAS 버전, 절대경로, 암호화 정보(Key, Salt 등) 노출)			
	취약점	**[IW-26] 에러처리 미흡**	위험도	하
	파라미터 변조 또는 비 정상적인 URL 요청으로 인해 발생하는 에러로 인해 정보가 노출될 경우 취약 (절대경로 정보, WEB, DB, WAS 정보 등)			
	취약점	**[IW-27] 임시/백업 파일 노출**	위험도	하
	웹 루트 하위에 index1.html, temp.html, test.html, backup.zip 등 서비스 운영에 사용하지 않는 파일이 존재하거나, 웹 어플리케이션에서 제공하는 기본 페이지, 설치 페이지, 샘플 페이지 등 불필요한 페이지가 존재할 경우 취약			
식별 오류	취약점	**[IW-28] 세션 예측**	위험도	중
	인증 후 부여되는 세션 및 토큰 값이 타인에 유추 가능한 값으로 부여되는 경우 취약			
	취약점	**[IW-29] 불충분한 세션 만료**	위험도	하
	최초 로그인 후 30 분 이상 Action 이 없을 경우에도 자동 로그아웃이 적용되지 않을 시 취약			
	취약점	**[IW-30] 세션 고정**	위험도	하
	특정 사용자 및 단말기에 고정된 값을 세션 및 토큰 값으로 사용하는 경우 취약			
	취약점	**[IW-31] 쿠키 변조**	위험도	중
	쿠키 사용 시 인증을 위한 중요정보가 안전한 알고리즘을 사용하지 않을 경우 취약			
암호화 오류	취약점	**[IW-32] 데이터 평문 전송**	위험도	중
	중요 정보 송·수신 구간에 SSL 암호화 통신이 적용되지 않을 경우 취약			
	취약점	**[IW-33] 취약한 암호화 알고리즘 사용**	위험도	하
	암호화가 적용된 데이터 또는 파라미터 값이 복호화가 가능한 취약한 암호화 (URL 인코딩, 진법 변경, BASE64, MD5 등)기법을 사용할 경우 취약			
취약하고 오래된 요소	취약점	**[IW-34] 공개된 패키지 취약점 허용**	위험도	중
	웹 어플리케이션 내에 알려진 패키지 및 웹 에디터 사용 시, 해당 라이브러리에서 알려진 취약점이 웹 어플리케이션에도 존재할 경우 취약			

[표 5] 웹 취약점 진단 점검 항목

3. 수행 결과

3.1. 웹 서비스 취약점 점검 결과

3.1.1. 총평

Vuln-Camp 웹 애플리케이션을 대상으로 총 11건의 보안 취약점 진단을 수행한 결과, 위험도 기준으로 상 5건, 중 5건, 하 1건의 취약점을 확인하였음.

인증·인가 영역에서는 클라이언트 쿠키 값만으로 권한을 판단하는 구조적 결함과 관리자 기능에 대한 서버 측 권한 검증 미흡을 확인하였음. 특히 쿠키 값 변조를 통해 관리자 권한을 획득한 후 공지사항 작성 등 관리자 기능을 비인가 상태로 수행할 수 있음을 확인하였음.

입력값 검증 영역에서는 Stored XSS 및 SQL Injection 취약점을 확인하였으며, 이를 통해 스크립트 실행 및 데이터베이스 질의 변조가 가능한 상태로 확인하였음. 또한 파일 업로드, 파일 다운로드, SSRF, CSRF 등 주요 기능에서 입력값 및 요청 검증이 미흡하여 서버 내부 자원 접근, 임의 파일 실행, 비밀번호 변경 등의 보안 위협이 발생할 수 있음을 확인하였음.

비즈니스 로직 영역에서는 결제 금액 파라미터 변조 및 클라이언트 측 쿠폰 검증 로직 노출 취약점을 확인하였음. 이를 통해 공격자가 결제 금액을 조작하거나 유효한 쿠폰 코드를 임의로 생성하여 금전적 피해로 이어질 가능성이 존재함.

전반적으로 서버 측 입력값 검증, 권한 검증, 업무 프로세스 검증이 미흡하며, 개발 편의를 위해 남겨진 디버그 기능 및 불필요한 정보 노출로 인해 후속 공격에 활용될 수 있는 정보가 노출되고 있어 조속한 보안 조치가 필요함.

3.1.2. 결과 요약

웹 서비스에서 발견된 취약점을 요약하면 다음과 같음

번호	서비스명	취약점 명	확인된 보안 위협	위험도
1	Vuln-Camp	[IW-03] SQL Injection	검색 파라미터 조작을 통해 DB 질의 변조 및 데이터 추출 가능	상
		[IW-11] Server-Side Request Forgery (SSRF)	사용자 입력 URL을 서버가 요청하여 내부망 및 로컬 리소스 접근 가능	상
		[IW-12] 파일 업로드	확장자 및 콘텐츠 검증 미흡으로 악성 파일 업로드 및 실행 가능	상
		[IW-13] 파일 다운로드	파일명 파라미터 조작으로 서버 내 임의 파일 다운로드 가능	상
		[IW-14] 프로세스 검증 누락	결제 금액 등 주요 파라미터를 클라이언트에서 조작하여 비정상 처리 가능	중
			쿠폰 생성·검증 로직이 클라이언트에 노출되어 할인 정책 우회 가능	중
		[IW-16] 불충분한 인가	쿠키 값 변조를 통해 관리자 권한 획득 후 공지사항 작성 가능	중
		[IW-17] Cross Site Scripting (XSS)	QnA 게시글 입력값 검증 미흡으로 스크립트 삽입 및 사용자 세션 탈취 가능	중
		[IW-18] Cross Site Request Forgery (CSRF)	CSRF 토큰 검증 미흡으로 사용자 의도와 무관한 비밀번호 변경 요청 가능	상
		[IW-24] 정보 누출	응답값 및 페이지 내 불필요한 시스템·계정 관련 정보 노출	중
		[IW-25] 서버 정보 노출	디버그 API를 통해 서버 내부 소스코드 및 구현 정보 확인 가능	하
		총 취약점 개수: 11개		

[표 6] 결과 요약

4. 취약점 진단 상세 내용

4.1. Vuln-Camp (127.0.0.1)

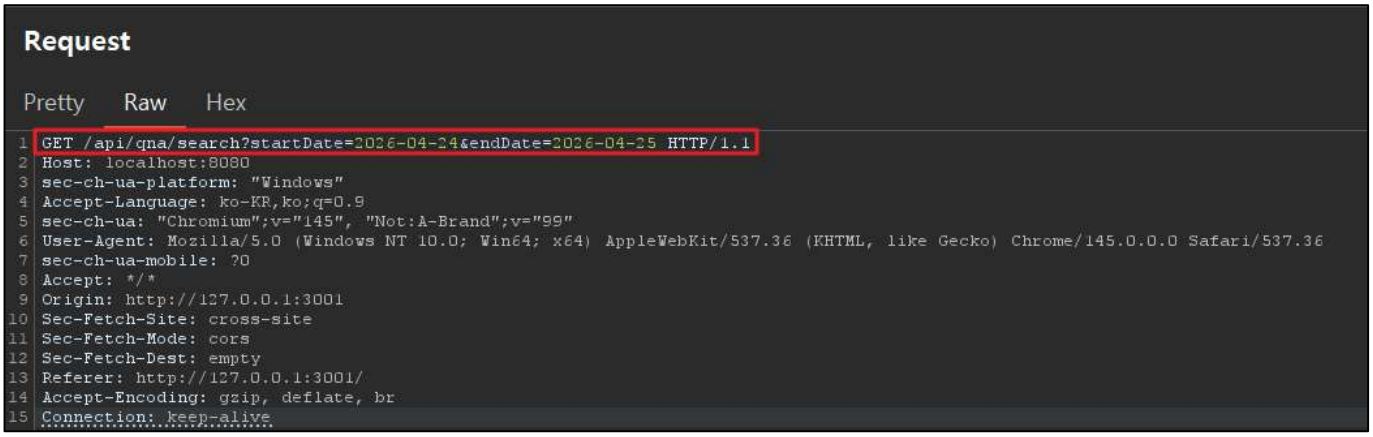
4.1.1. SQL Injection

Vuln-Camp 웹 애플리케이션의 QnA 게시판 날짜 검색 기능에서 사용자 입력값이 SQL 질의문에 안전하게 처리되지 않는 것을 확인하였음. 특히 startDate, endDate 파라미터 값 조작을 통해 SQL 조건절을 변경할 수 있으며, 이를 통해 공격자가 정상 검색 범위를 우회하여 QnA 게시글 정보를 비정상적으로 조회할 수 있음.

Vuln-Camp 웹 애플리케이션의 고객센터(QnA) 게시판에서 시작일과 종료일을 기준으로 게시글을 검색하는 기능이 존재하는 것을 확인하였음.

[그림 1] QnA 게시판 날짜 검색 기능 확인

QnA 게시판의 날짜 검색 기능을 수행한 결과, GET 방식으로 /api/qna/search 요청이 발생하며 startDate와 endDate 파라미터가 서버로 전달되는 것을 확인하였음.



[그림 2] QnA 게시판 날짜 검색 요청 파라미터 확인

endDate 파라미터에 참 조건에 해당하는 SQL 구문(' OR '1'='1)을 삽입하여 요청한 결과, 정상 날짜 범위에 해당하지 않음에도 QnA 게시글 목록이 조회되는 것을 확인하였음.

Request

Pretty Raw Hex

```
1  GET /api/qna/search?startDate=2099-01-01&endDate=2099-01-01'+OR+'1'%3d'1 HTTP/1.1
2  Host: localhost:8080
3  sec-ch-ua-platform: "Windows"
4  Accept-Language: ko-KR,ko;q=0.9
5  sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
6  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
7  sec-ch-ua-mobile: ?0
8  Accept: */*
9  Origin: http://127.0.0.1:3001
10 Sec-Fetch-Site: cross-site
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: http://127.0.0.1:3001/
14 Accept-Encoding: gzip, deflate, br
15 Connection: keep-alive
```

Search

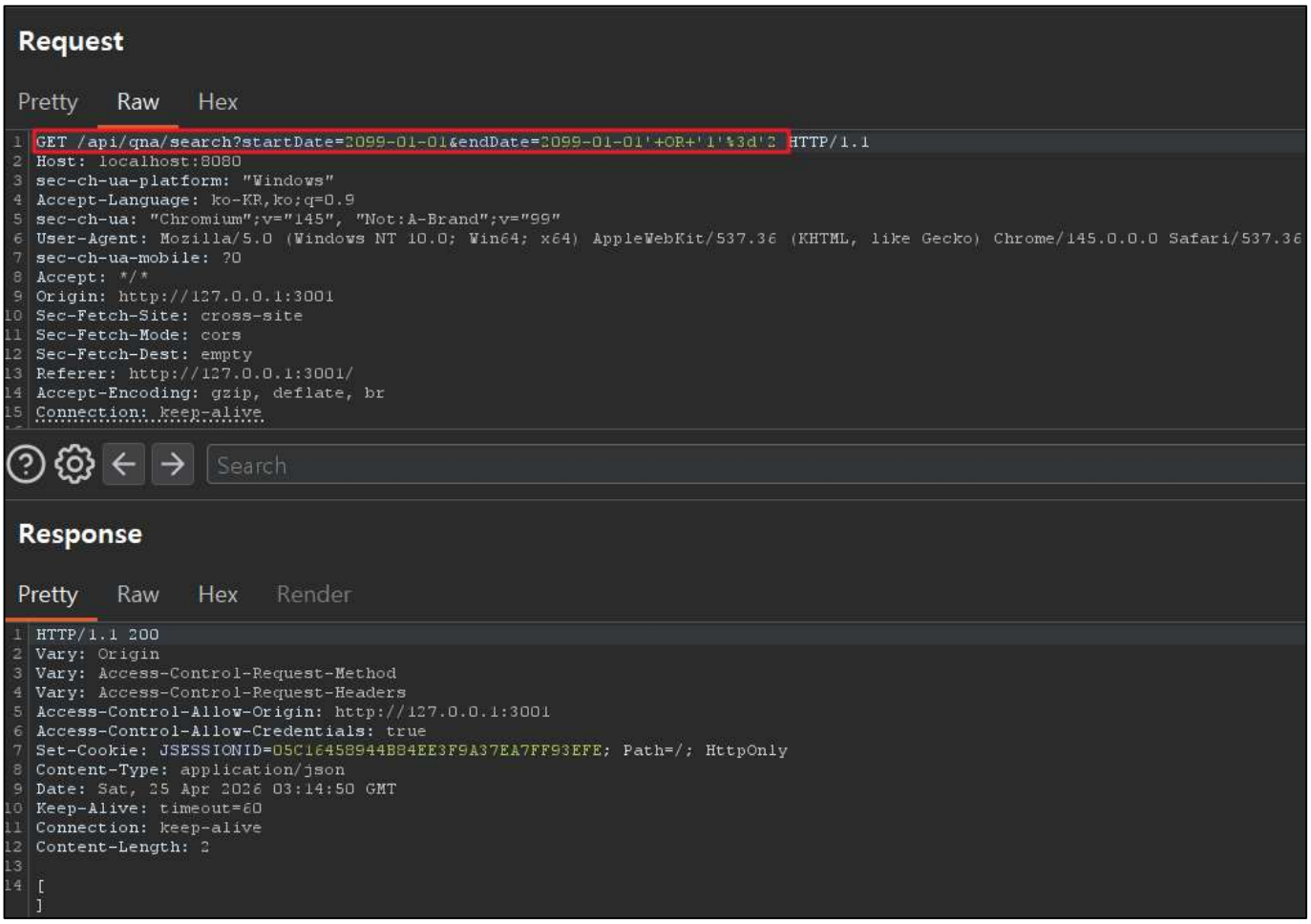
Response

Pretty Raw Hex Render

```
1  HTTP/1.1 200
2  Vary: Origin
3  Vary: Access-Control-Request-Method
4  Vary: Access-Control-Request-Headers
5  Access-Control-Allow-Origin: http://127.0.0.1:3001
6  Access-Control-Allow-Credentials: true
7  Set-Cookie: JSESSIONID=8086543513117EF03CA891DACF78978B; Path=/; HttpOnly
8  Content-Type: application/json
9  Date: Sat, 25 Apr 2026 03:15:17 GMT
10 Keep-Alive: timeout=60
11 Connection: keep-alive
12 Content-Length: 1570
13
14 [
       {
           "id":7,
           "title":"□□ □□□□ □□□ □□□",
           "content":"□ □□□□ □□□□□ □□ Path Traversal □ □□□□ □□□□ □□□□ □ □□□□.",
           "author":"admin@vuln.camp",
           "status":"Completed",
           "answer":null,
           "createdAt":"2026-04-25T11:58:47.047892",
           "attachmentPath":"test-download.txt"
       },
       {
           "id":5,
           "title":"□□□ □□ □□□□□",
           "content":"□□ □□□□□□. (5)",
           "author":"user1",
           "status":"Pending",
           "answer":null,
           "createdAt":"2026-04-25T11:58:47.045689",
           "attachmentPath":null
       },
       {
           "id":6,
```

[그림 3] 참 조건 SQL Injection 구문 삽입 결과

동일한 날짜 조건에서 endDate 파라미터에 거짓 조건에 해당하는 SQL 구문(' OR '1'='2)을 삽입하여 요청한 결과, QnA 게시글 목록이 조회되지 않는 것을 확인하였음.



[그림 4] 거짓 조건 SQL Injection 구문 삽입 결과

4.1.2. SSRF

Vuln-Camp 웹 애플리케이션의 소스코드 노출을 통해 /api/ssrf/fetch API가 url 파라미터를 입력받아 서버 측 요청을 수행하는 기능임을 확인하였음. 해당 API는 입력 URL에 대한 검증이 미흡하여 공격자가 지정한 내부망 또는 로컬 주소로 서버 측 요청을 유도할 수 있으며, 이를 통해 내부 서비스 응답 정보가 비정상적으로 노출될 수 있음.

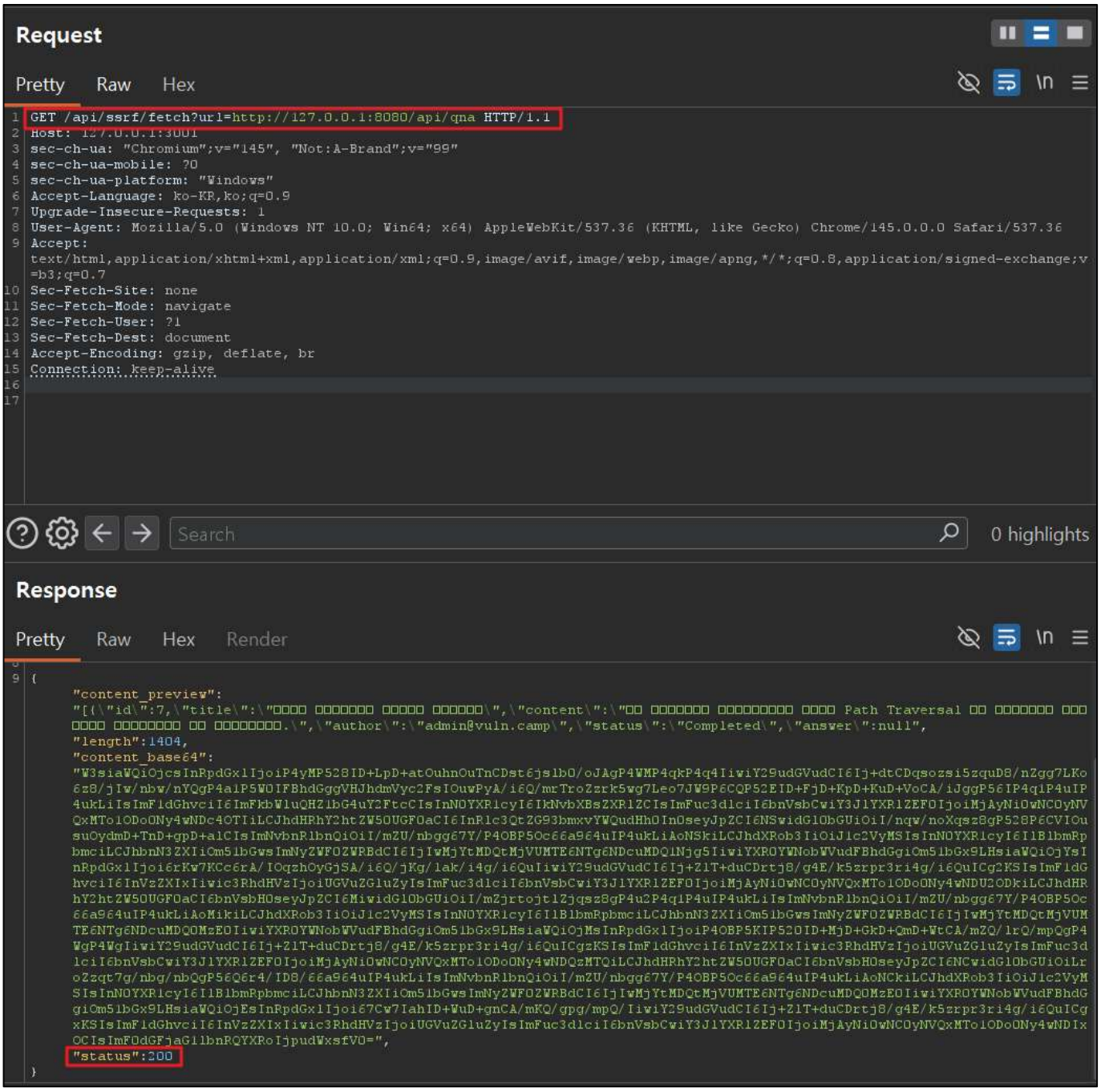
소스코드 노출이 존재하는 /api/debug/source 기능을 이용하여 SSRFController.java 파일을 조회한 결과, /api/ssrf 경로의 컨트롤러가 존재하며 하위 /fetch 경로에서 URL 파라미터를 입력 받는 기능이 구현되어 있는 것을 확인하였음.

요청 URL: /api/debug/source?file=src/main/java/com/vulnlab/web/controller/SSRFController.java

```
package com.vulnlab.web.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.Base64;
import java.util.HashMap;
import java.util.Map;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/ssrf")
public class SSRFController {

    // ►► ►섛씠吏► 誘멷ㅡ蹂닿린 (Proxy 澑곕뒬)
    @GetMapping("/fetch")
    public ResponseEntity<?> fetchUrl(@RequestParam String url) {
        try {
            URL targetUrl = new URL(url);
            HttpURLConnection connection = (HttpURLConnection) targetUrl.openConnection();
            connection.setRequestMethod("GET");

            int status = connection.getResponseCode();

            // ►뮜떟 ►곗씠►► ►쎄린
            BufferedReader in = new BufferedReader(new InputStreamReader(connection.getInputStream()));
            String content = in.lines().collect(Collectors.joining("₩n"));
            in.close();

            Map<String, Object> response = new HashMap<>();
            response.put("status", status);
            response.put("length", content.length());
            // JSON ►뚯떛 ►ㅛ쩋 誤⑥►瑜► ►꽃빐 Base64 ►몄팸►► 泥섚ㅡ
            response.put("content_preview", content.substring(0, Math.min(content.length(), 200)));
            response.put("content_base64", Base64.getEncoder().encodeToString(content.getBytes()));

            return ResponseEntity.ok(response);

        } catch (Exception e) {
            return ResponseEntity.badRequest().body("Error fetching URL: " + e.getMessage());
        }
    }
}
```

[그림 5] 소스코드 노출을 통한 SSRF 컨트롤러 확인

확인된 /api/ssrf/fetch API의 url 파라미터에 로컬 내부 API 주소(http://127.0.0.1:8080/api/qna)를 입력하여 요청한 결과, 서버가 해당 주소로 직접 요청을 수행하고 content_preview 및 content_base64 항목을 통해 내부 API 응답 정보를 반환하는 것을 확인하였음.

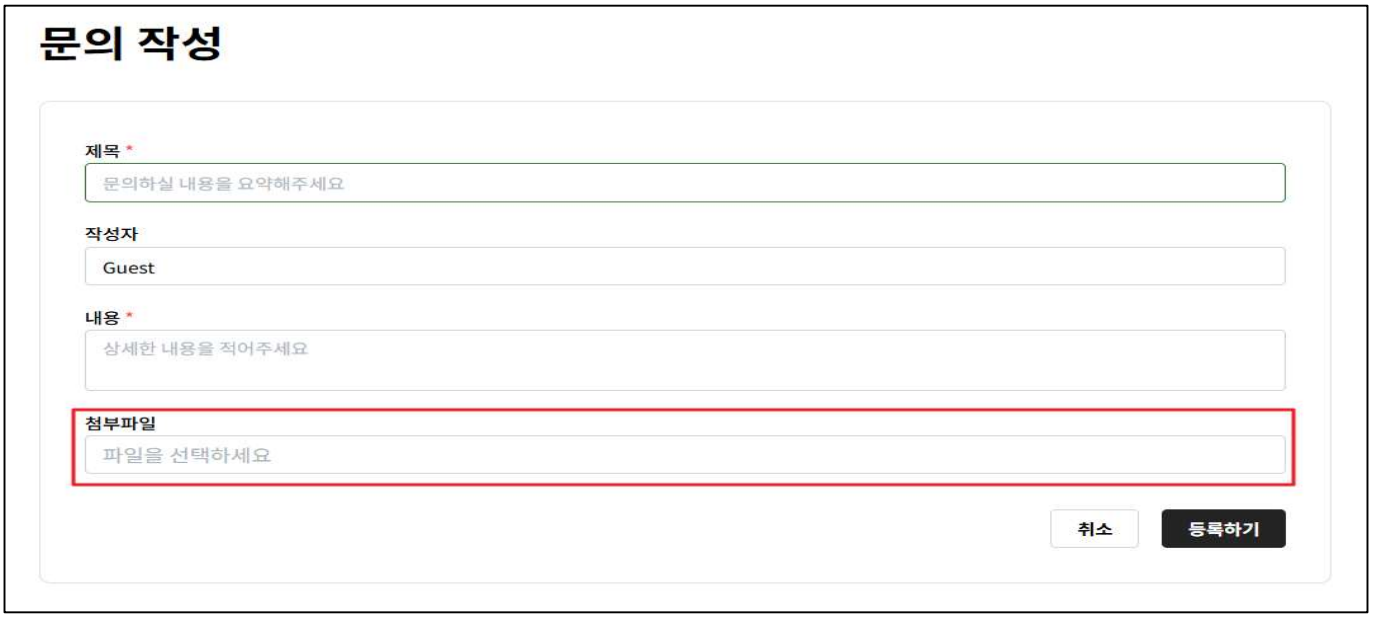


[그림 6] SSRF를 통한 내부 API 응답 정보 확인

4.1.3. 파일 업로드

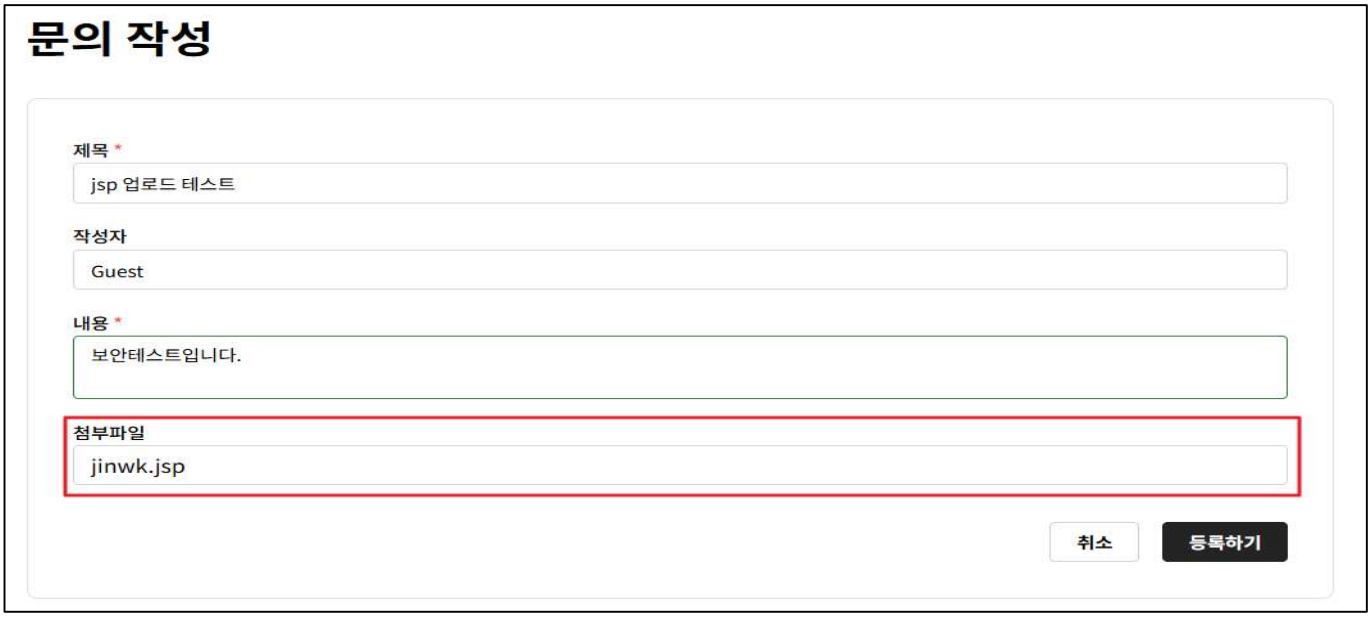
Vuln-Camp 웹 애플리케이션의 QnA 게시글 작성 기능에서 첨부파일 업로드 시 파일 확장자 및 콘텐츠 검증이 미흡한 것을 확인하였음. 이를 통해 공격자가 서버 사이드 스크립트 파일(JSP 등)을 업로드할 수 있으며, 업로드된 파일이 서버에서 실행될 경우 웹셸 실행 및 서버 명령 수행 등의 보안 위협이 발생할 수 있음.

QnA 게시글 작성 기능에서 첨부파일을 업로드할 수 있는 입력 항목이 존재하는 것을 확인하였음.



[그림 7] QnA 게시글 첨부파일 업로드 기능 확인

첨부파일 업로드 기능을 통해 서버 사이드 스크립트 파일인 JSP 파일 업로드를 시도하였음.



[그림 8] JSP 파일 업로드 요청

파일 확장자 및 콘텐츠 검증 없이 JSP 파일이 업로드 되는 것을 확인하였음.

[그림 9] JSP 파일 업로드 성공 확인

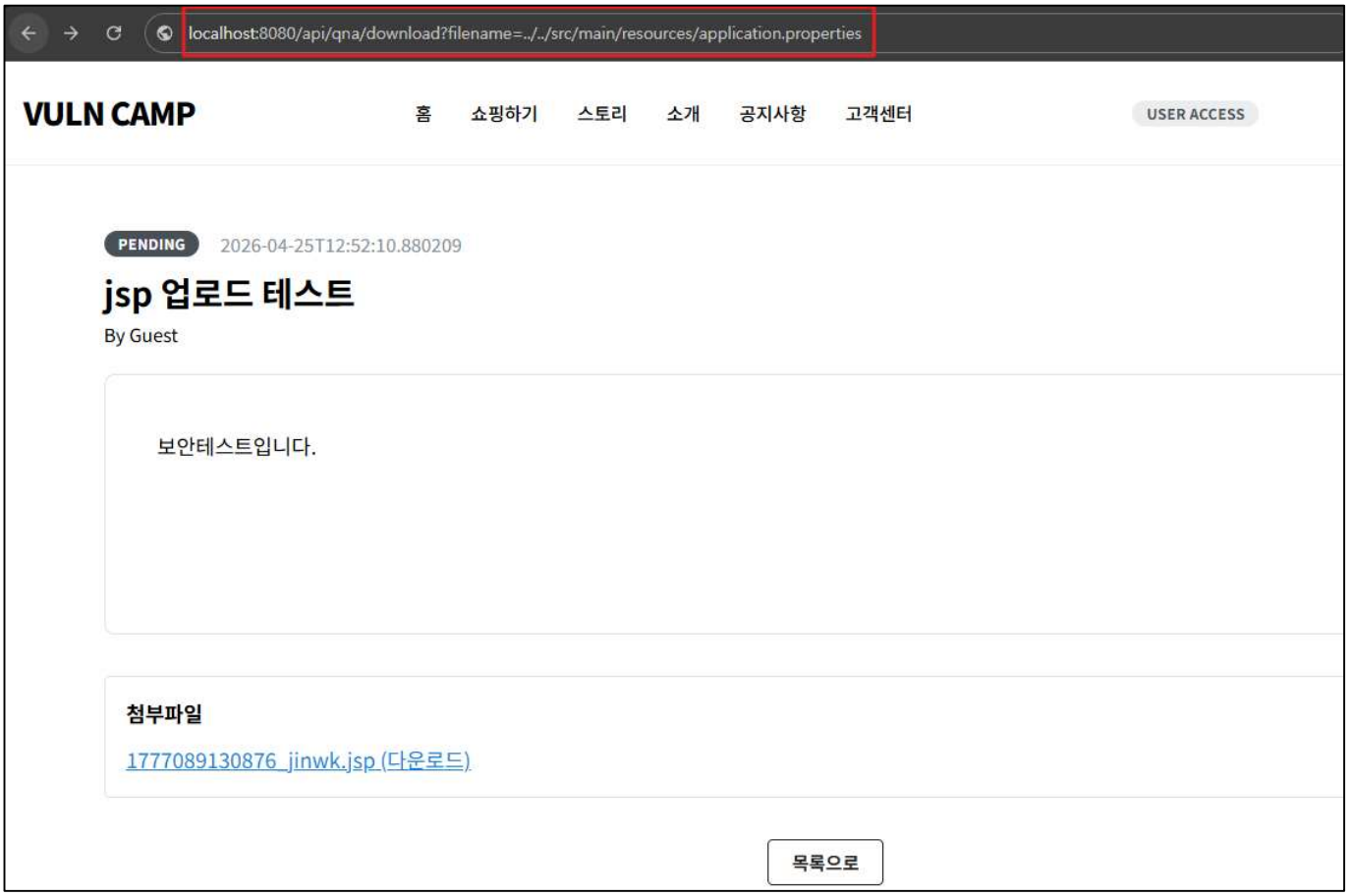
QnA 첨부파일 업로드 기능을 통해 업로드한 JSP 파일에 직접 접근한 결과, JSP 코드 실행 결과인 JSP_JINWK_TEST 문자열이 응답되는 것을 확인하였음.

[그림 10] JSP 파일 업로드 후 서버 측 실행 확인

4.1.4. 파일 다운로드

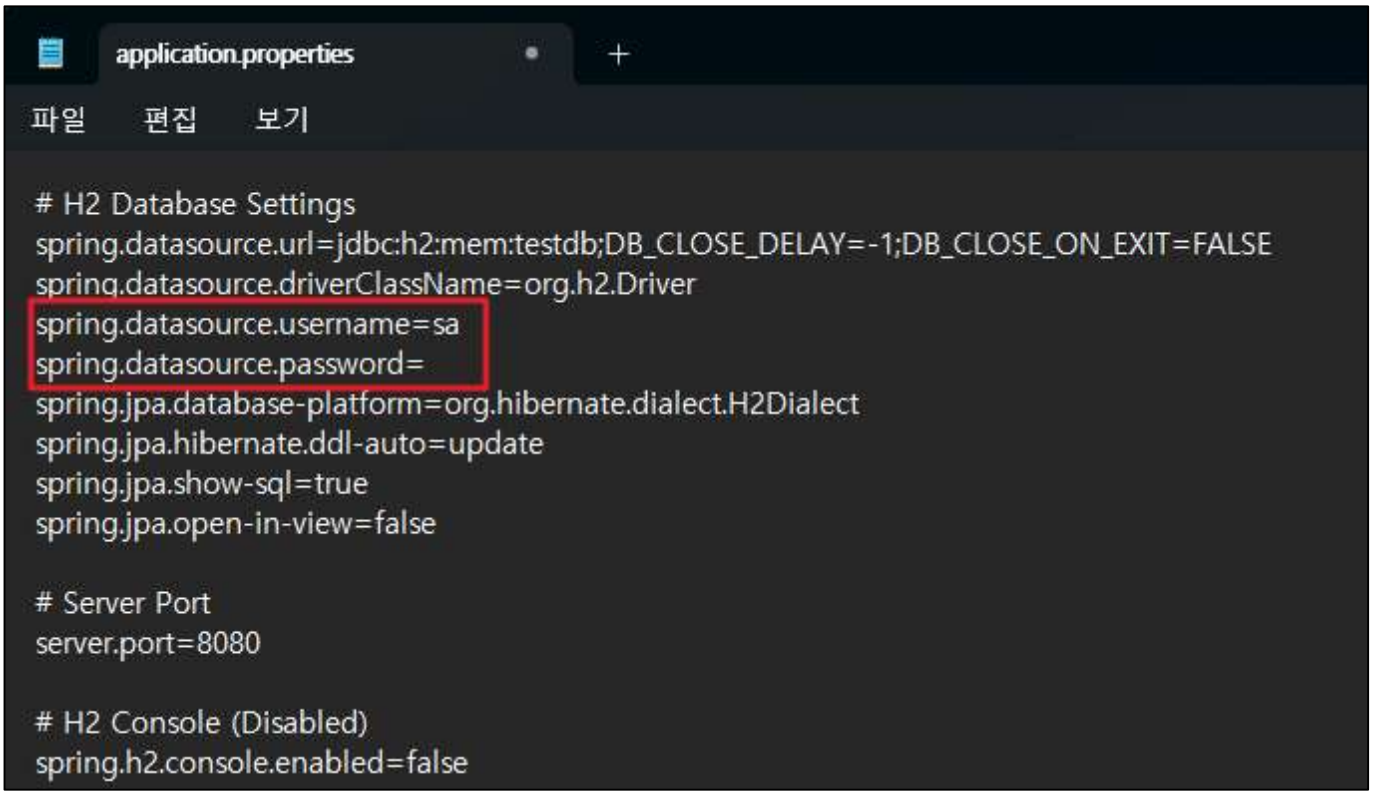
filename 파라미터에 상위 디렉터리 이동 구문(../)을 삽입하여 서버 내부 설정 파일 다운로드 가능 여부를 확인하였음. 요청 결과, 서버 내부 설정 파일인 application.properties가 다운로드되었으며, 데이터베이스 설정 정보 및 서버 포트 등 주요 설정 정보가 노출되는 것을 확인하였음.

QnA 첨부파일 다운로드 요청에서 filename 파라미터 값을 ../src/main/resources/application.properties로 조작하여 서버 내부 설정 파일 다운로드를 시도하였음.



[그림 11] filename 파라미터 경로 조작

filename 파라미터 조작 결과, 서버 내부 설정 파일인 application.properties가 다운로드 되어 DB 접속 정보 및 서버 설정 정보가 노출되는 것을 확인하였음.

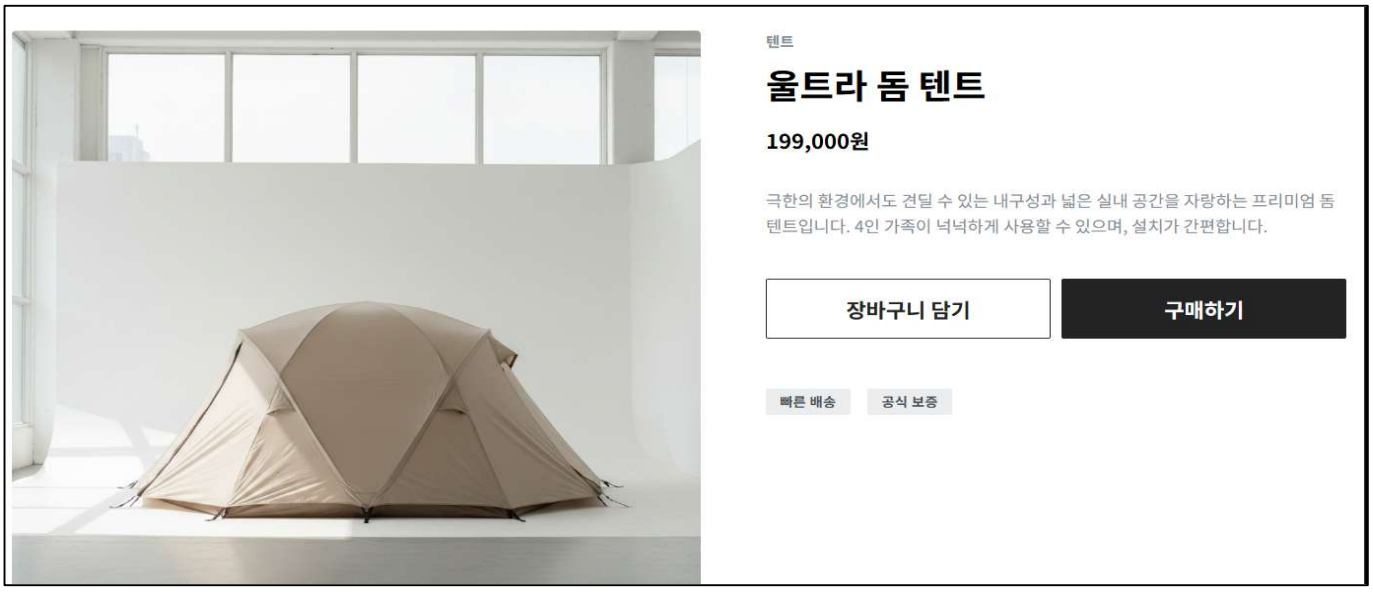


[그림 12] 경로 조작을 통한 서버 내부 설정 파일 다운로드 확인

4.1.5. 프로세스 검증 누락

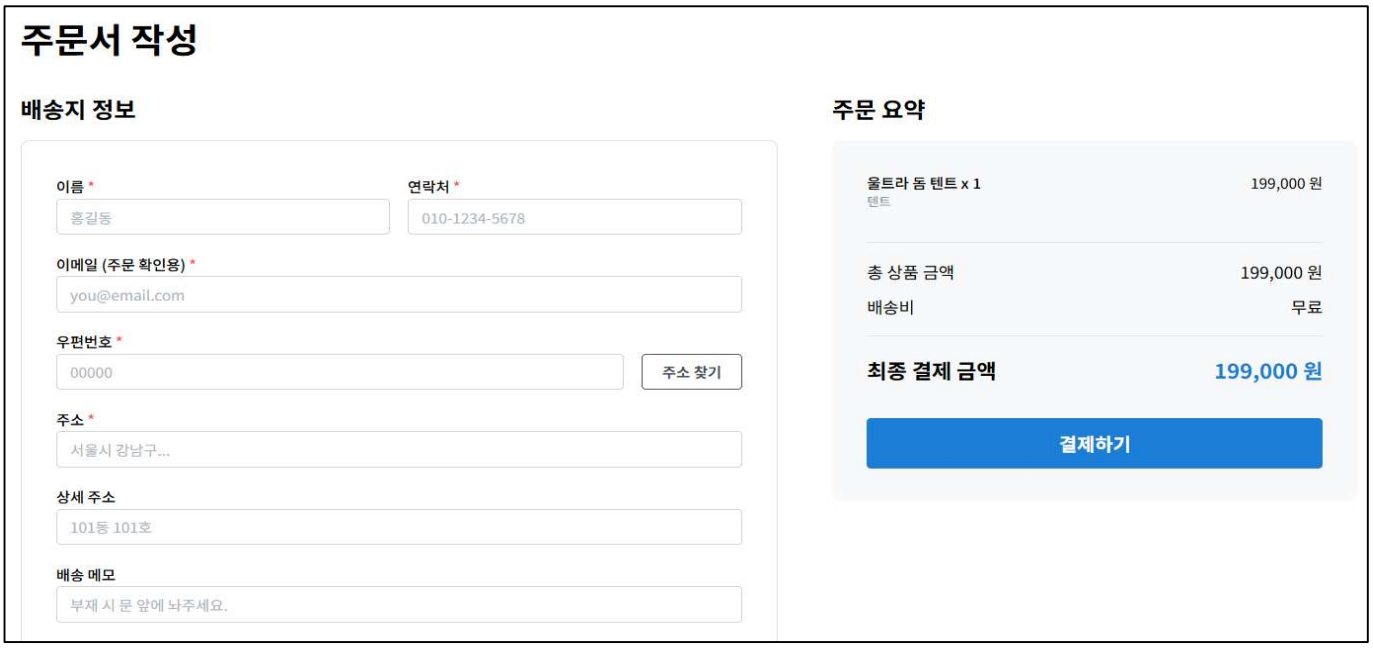
Vuln-Camp 웹 애플리케이션의 결제 금액 처리 및 쿠폰 검증 기능에서 서버 측 프로세스 검증이 미흡한 것을 확인하였음. 해당 기능은 클라이언트 측 요청값 또는 로직에 대한 서버 측 검증이 충분히 수행되지 않아, 공격자가 정상 업무 절차를 우회하거나 조작된 값으로 처리를 수행할 수 있음.

상품 상세 페이지에서 울트라 돔 텐트의 정상 판매 금액이 199,000원으로 표시되는 것을 확인하였음.



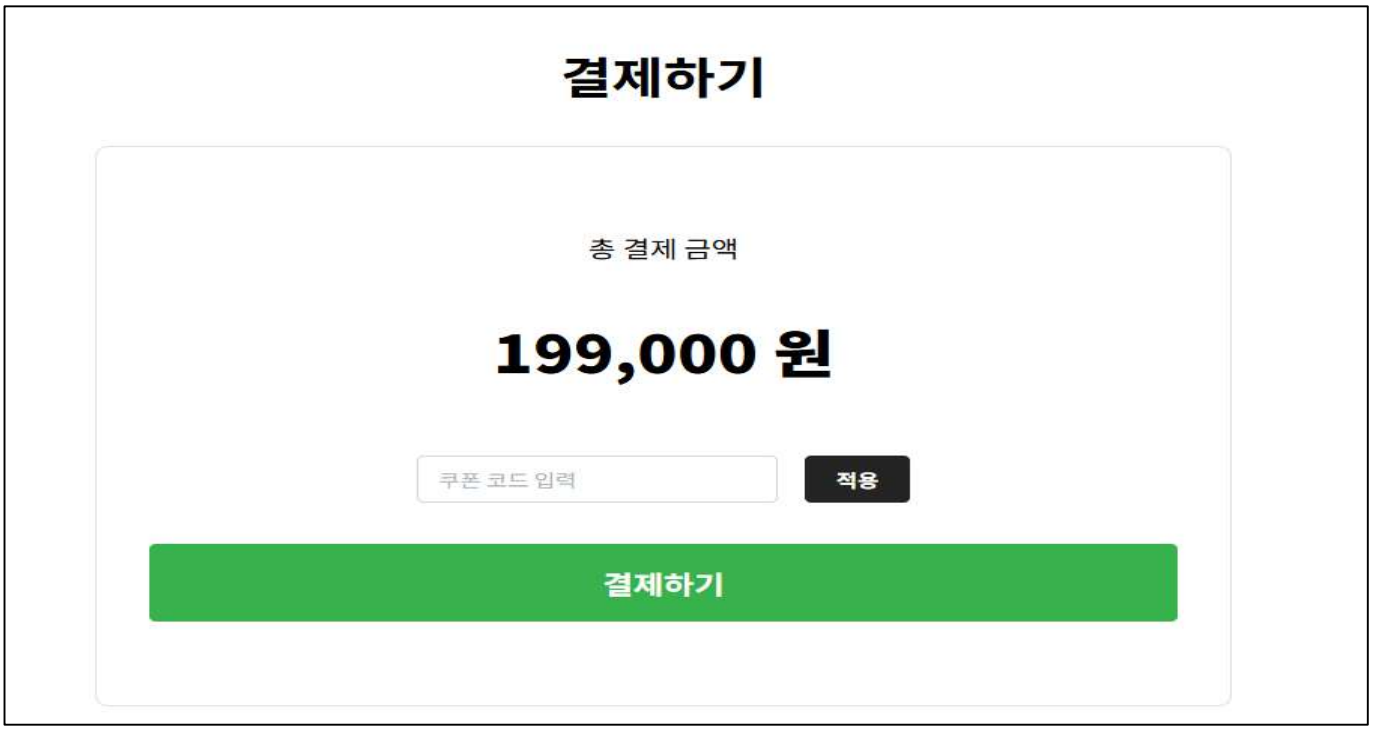
[그림 13] 상품 상세 페이지 정상 금액 확인

구매하기 버튼 클릭 후 이동한 주문서 작성 페이지에서도 주문 요약 내 최종 결제 금액이 199,000원으로 표시되는 것을 확인하였음.



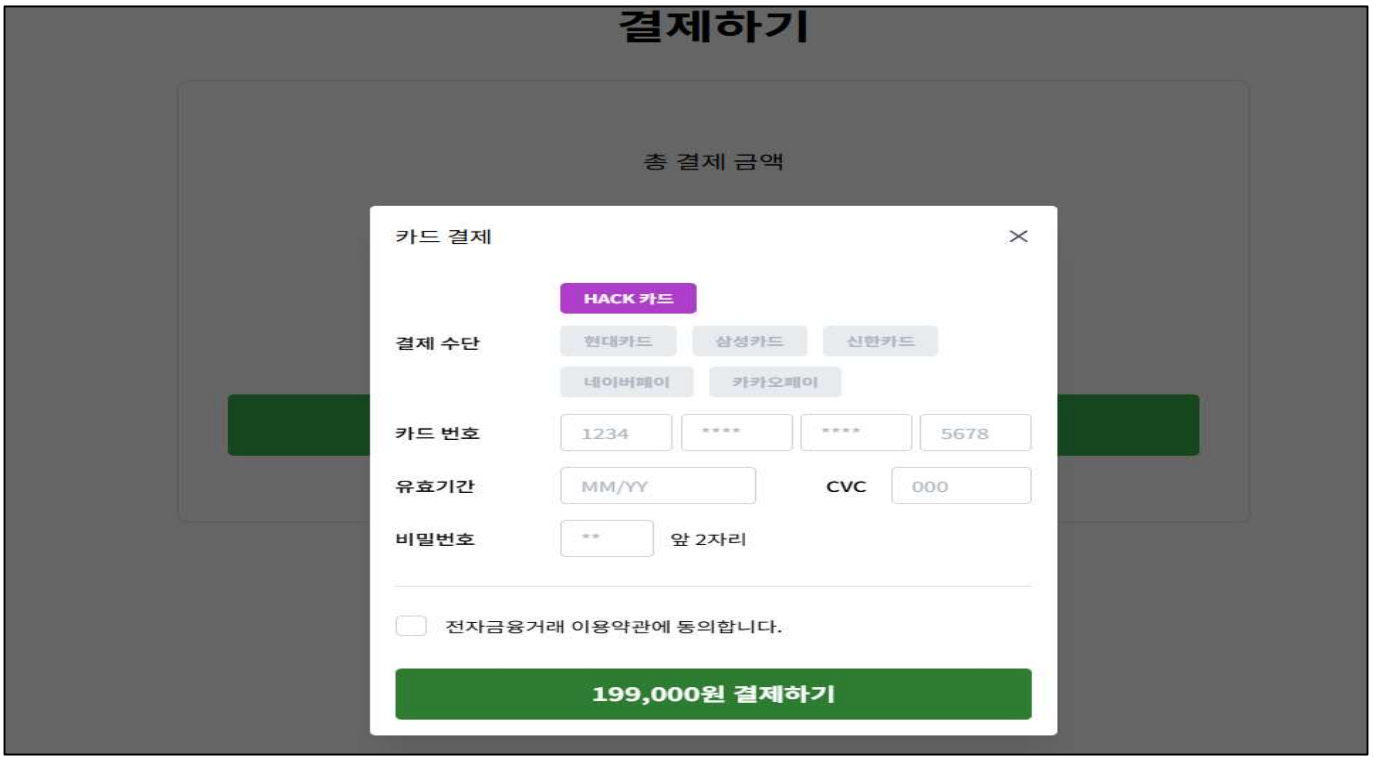
[그림 14] 주문서 작성 페이지 정상 금액 확인

결제 페이지에서도 총 결제 금액이 199,000원으로 표시되며, 카드 결제 요청을 통해 주문이 진행되는 것을 확인하였음.



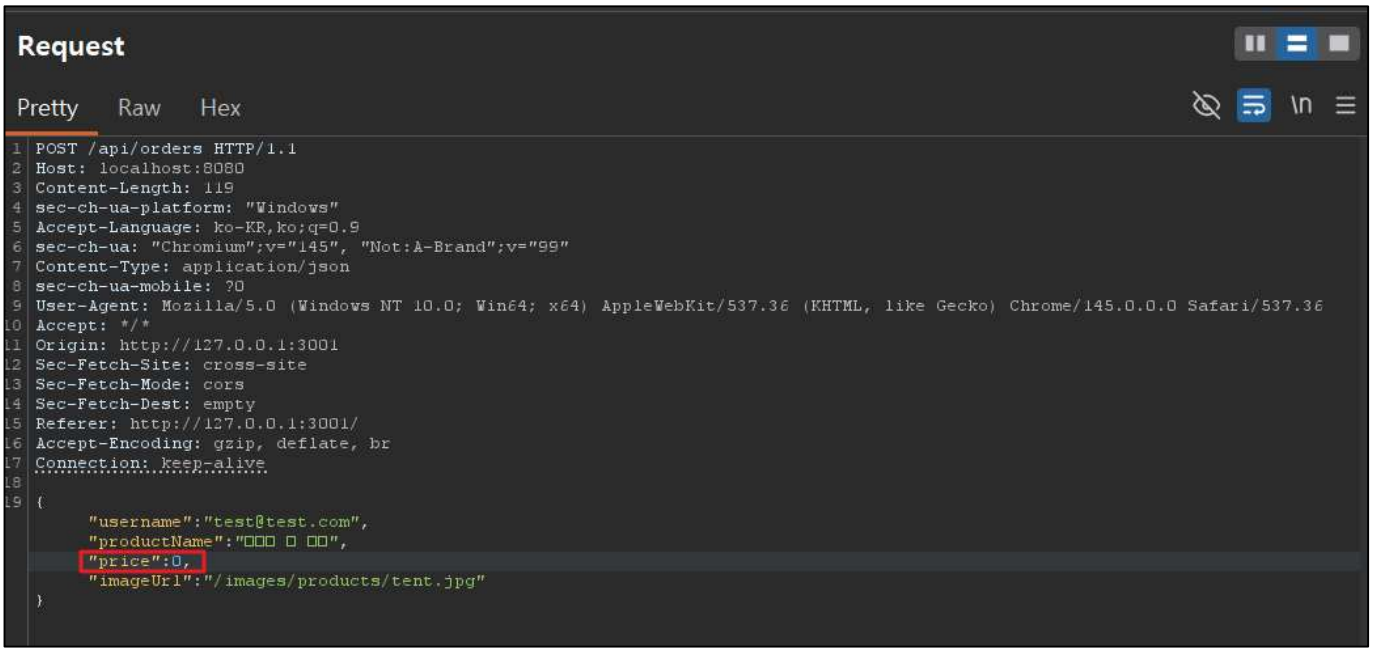
[그림 15] 결제 페이지 정상 금액 확인

결제 페이지에서 카드 결제 모달을 호출한 결과, 결제 버튼에 정상 결제 금액인 199,000원이 표시되는 것을 확인하였음.



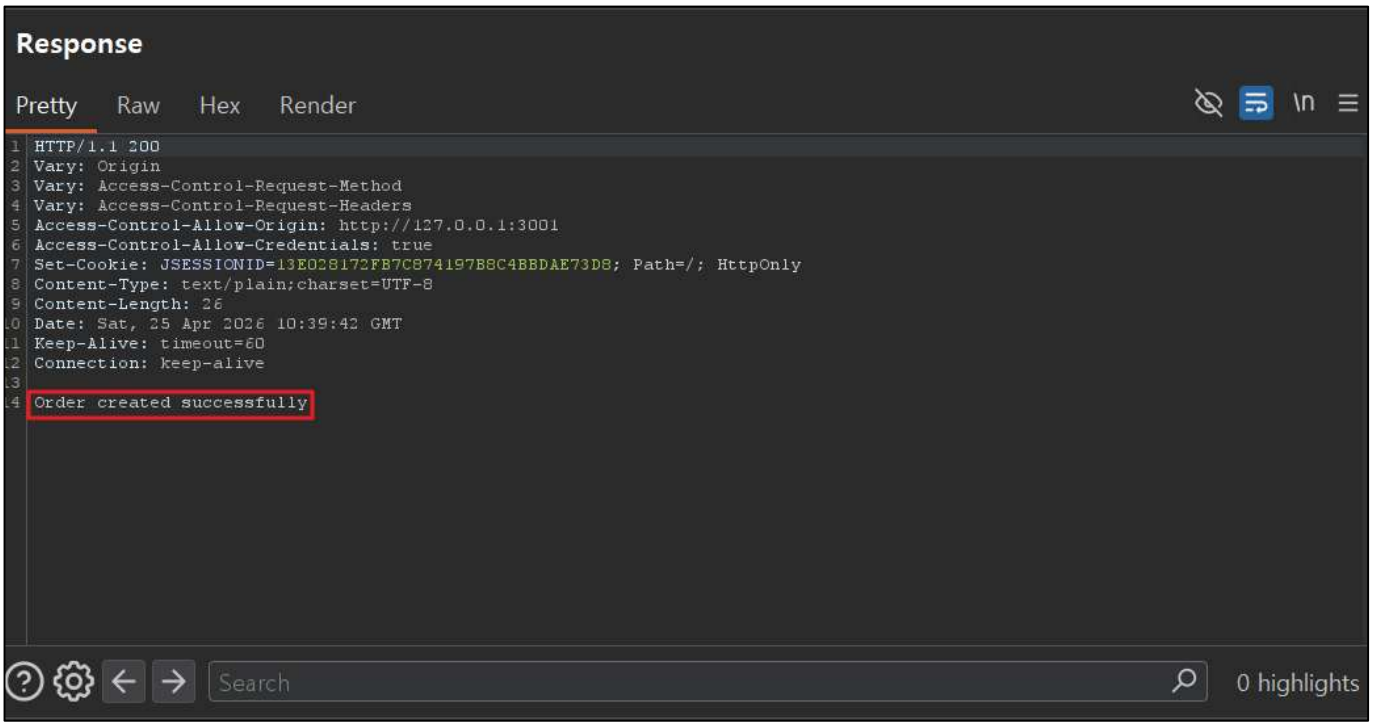
[그림 16] 카드 결제 모달 정상 금액 확인

주문 생성 요청을 확인한 결과, 상품 가격 정보가 price 파라미터로 클라이언트에서 서버로 전달되는 것을 확인하였음. 해당 요청에서 price 값을 0으로 변조하여 서버로 전송하였음.



[그림 17] 결제 금액 파라미터 변조 요청

price 값을 0으로 변조한 주문 생성 요청에 대해 서버가 별도 검증 없이 정상 응답을 반환하는 것을 확인하였음.

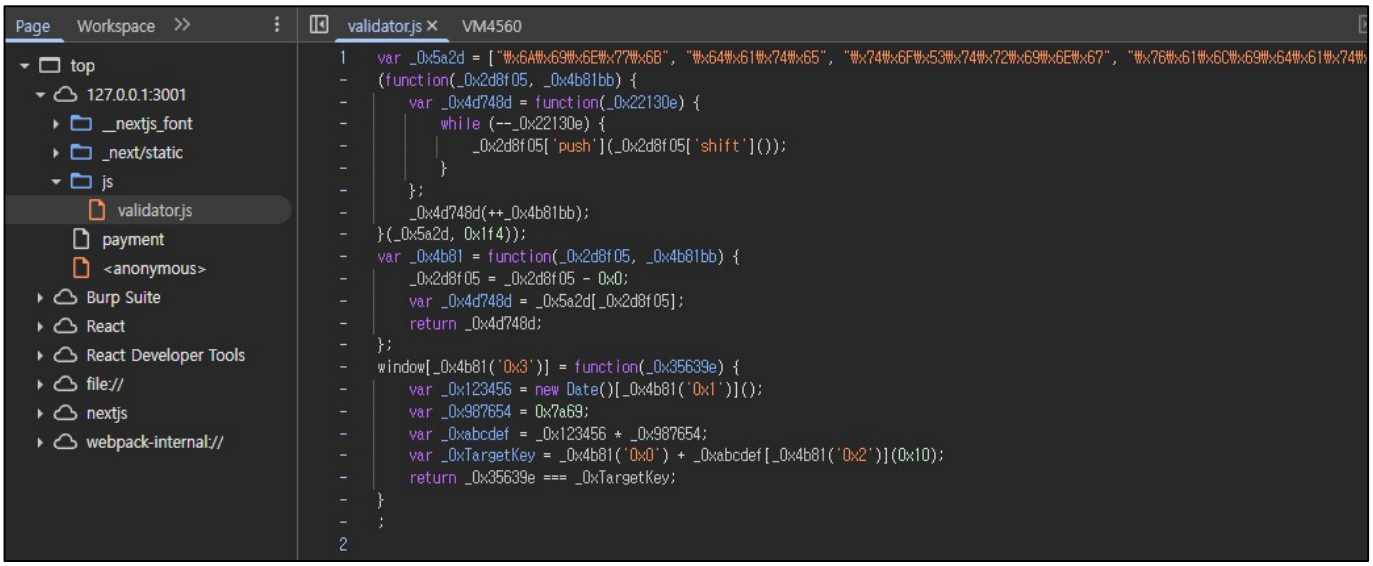


[그림 18] 변조된 결제 금액 주문 생성 응답 확인

주문 내역을 확인한 결과, 변조된 price 값이 실제 주문 결과에 반영되어 결제 금액이 0원으로 표시되는 것을 확인하였음.

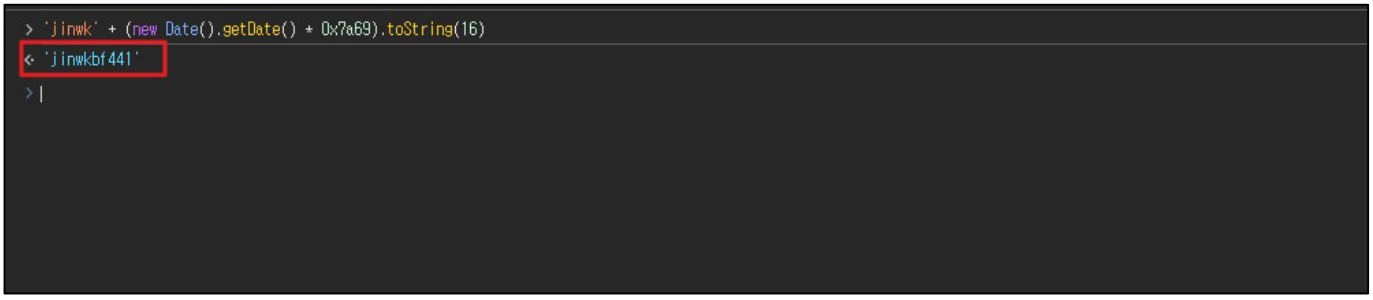
[그림 19] 변조된 결제 금액 주문 내역 반영 확인

브라우저 개발자도구를 통해 클라이언트 측 JavaScript 파일을 확인한 결과, 쿠폰 검증 로직이 validator.js 파일에 포함되어 있는 것을 확인하였음.



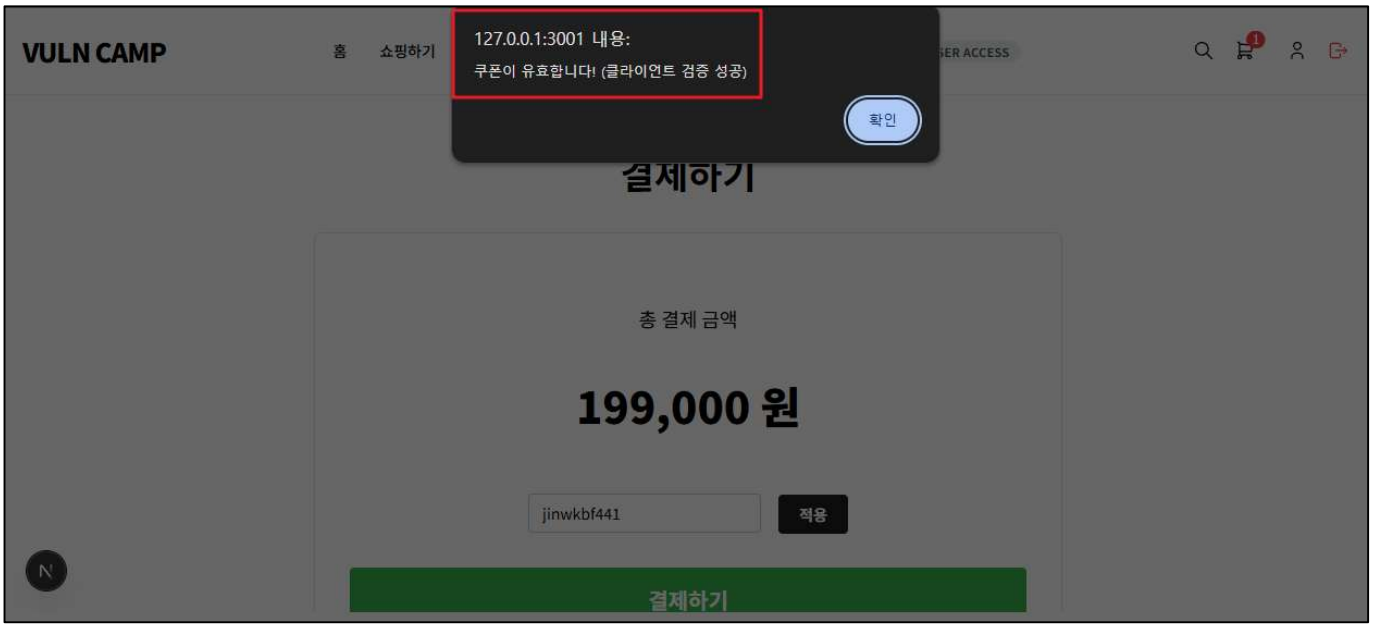
[그림 20] 클라이언트 측 쿠폰 검증 로직 노출 확인

노출된 쿠폰 검증 로직을 분석하여 쿠폰 생성 규칙을 확인하였으며, 브라우저 콘솔에서 유효한 쿠폰 코드를 생성하였음.



[그림 21] 쿠폰 생성 규칙 분석 및 유효 쿠폰 코드 생성

생성한 쿠폰 코드를 결제 페이지에 입력한 결과, 클라이언트 측 검증을 통해 쿠폰이 유효한 것으로 처리되는 것을 확인하였음.



[그림 22] 유효 쿠폰 코드 적용 확인

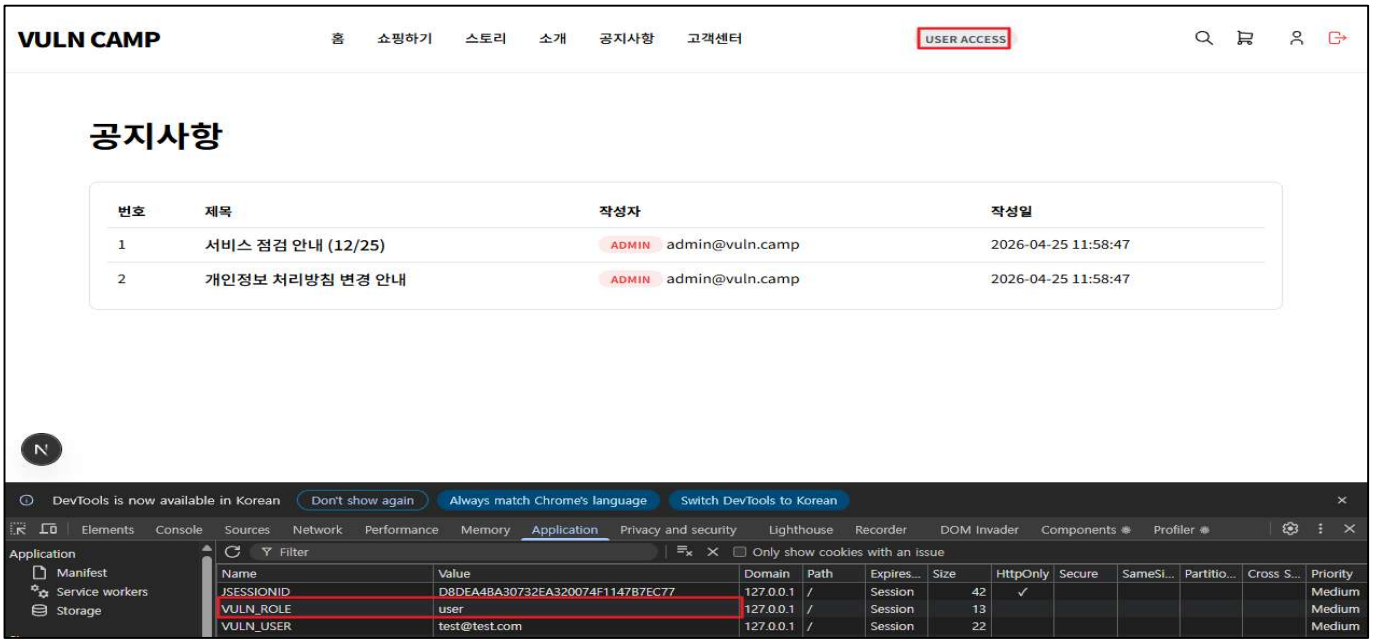
쿠폰 적용 후 주문을 완료한 결과, 주문 내역에서 할인된 결제 금액이 반영되는 것을 확인하였음.

[그림 23] 쿠폰 적용 후 할인 금액 반영 확인

4.1.6. 불충분한 인가

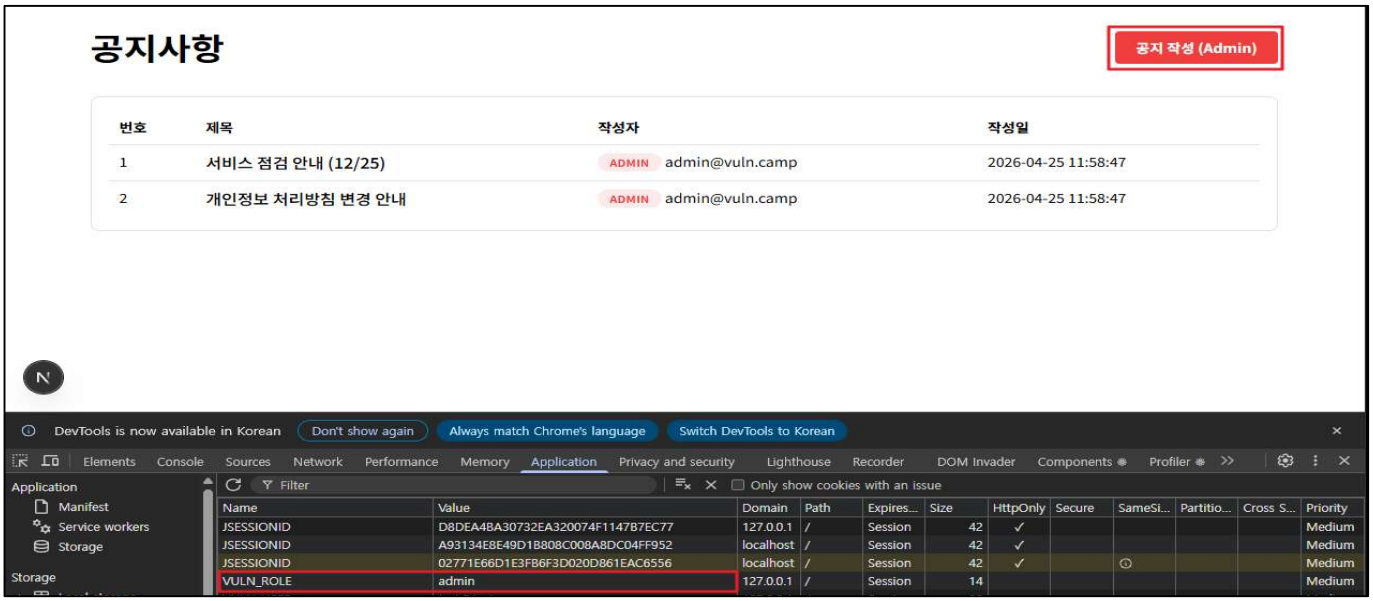
Vuln-Camp 웹 애플리케이션의 관리자 기능 접근 과정에서 서버 측 권한 검증이 미흡한 것을 확인하였음. 특히 클라이언트에 저장된 VULN_ROLE 쿠키 값을 admin으로 변조할 경우 관리자 권한으로 인식되며, 이를 통해 일반 사용자도 관리자 권한이 필요한 공지사항 작성 기능을 수행할 수 있음.

일반 사용자 상태에서는 공지사항 목록 화면에서 관리자 기능인 공지사항 작성 버튼이 노출되지 않는 것을 확인하였음.



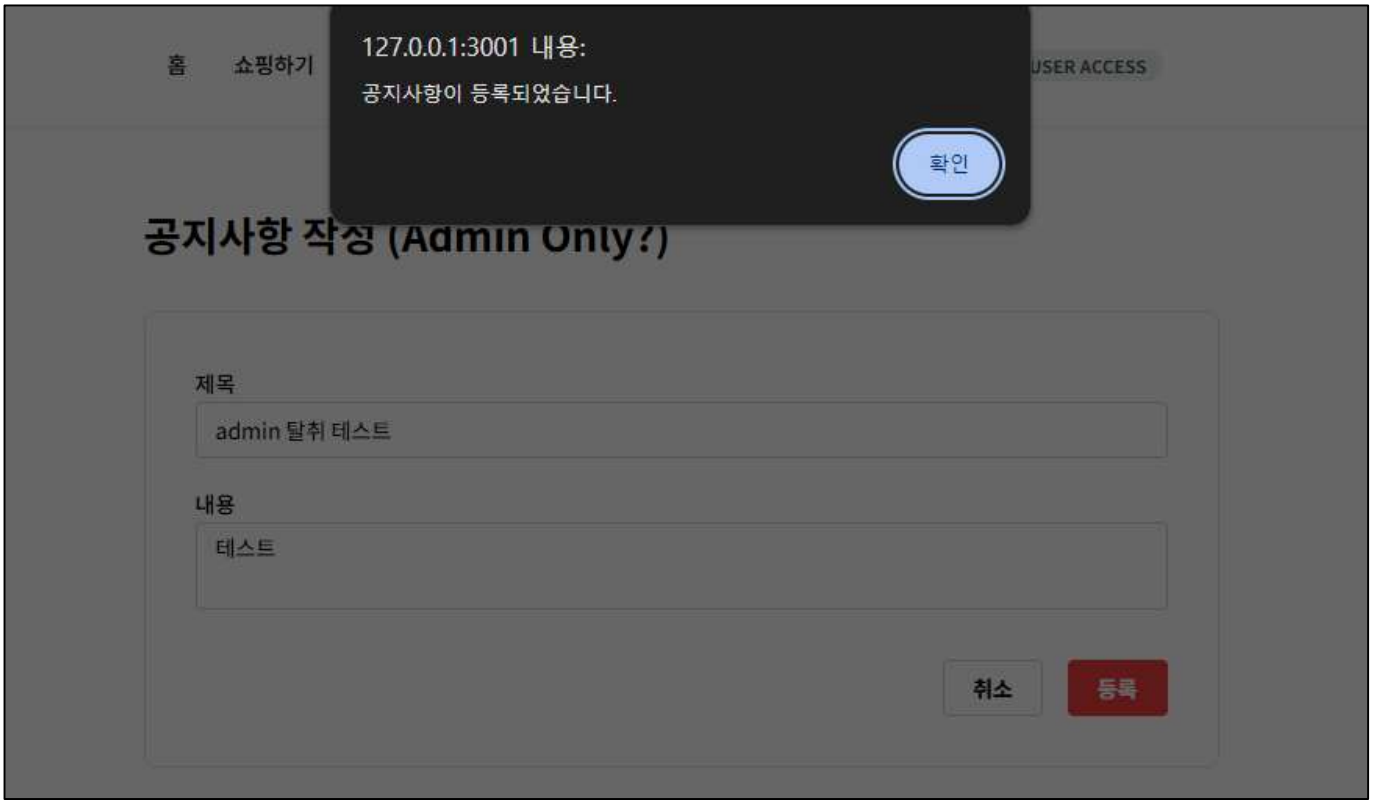
[그림 24] 일반 사용자 상태 공지사항 화면 확인

브라우저 개발자도구에서 VULN_ROLE 쿠키 값을 admin으로 변조한 후 공지사항 작성 버튼이 노출되는 것을 확인하였음.



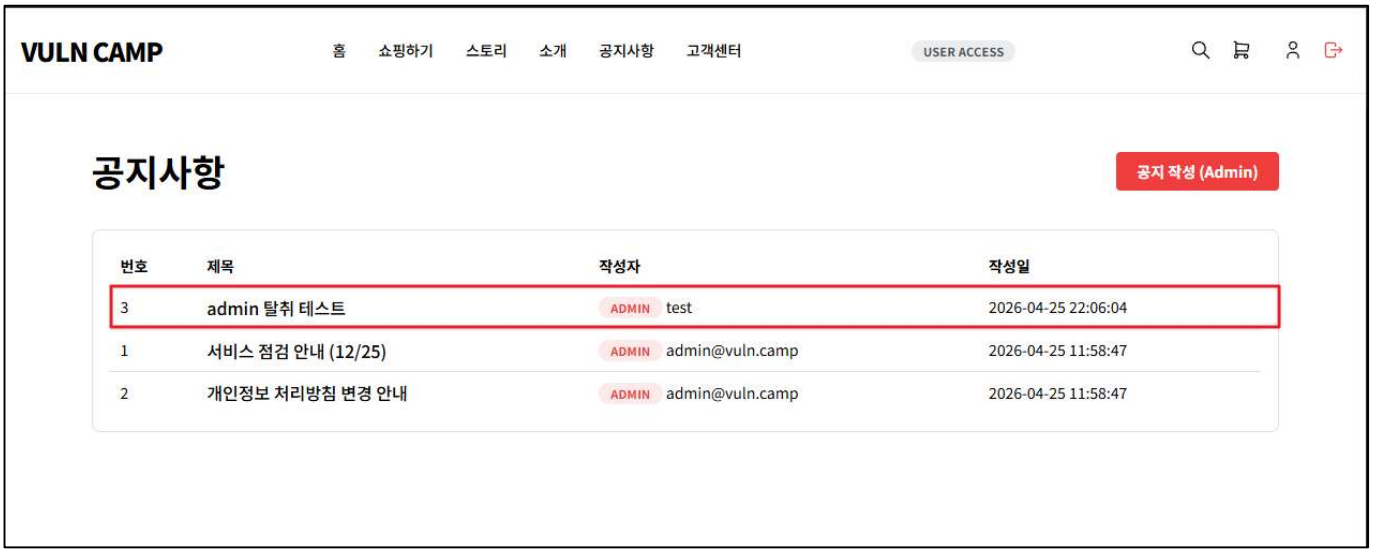
[그림 25] 쿠키 변조를 통한 관리자 기능 노출 확인

쿠키 값 변조 후 관리자 권한이 필요한 공지사항 작성 페이지에 접근하여 공지사항 등록을 시도하였음.



[그림 26] 관리자 공지사항 작성 요청

공지사항 등록 요청 결과, 일반 사용자가 쿠키 값 변조를 통해 관리자 권한 기능을 수행하고 공지사항이 정상 등록되는 것을 확인하였음.



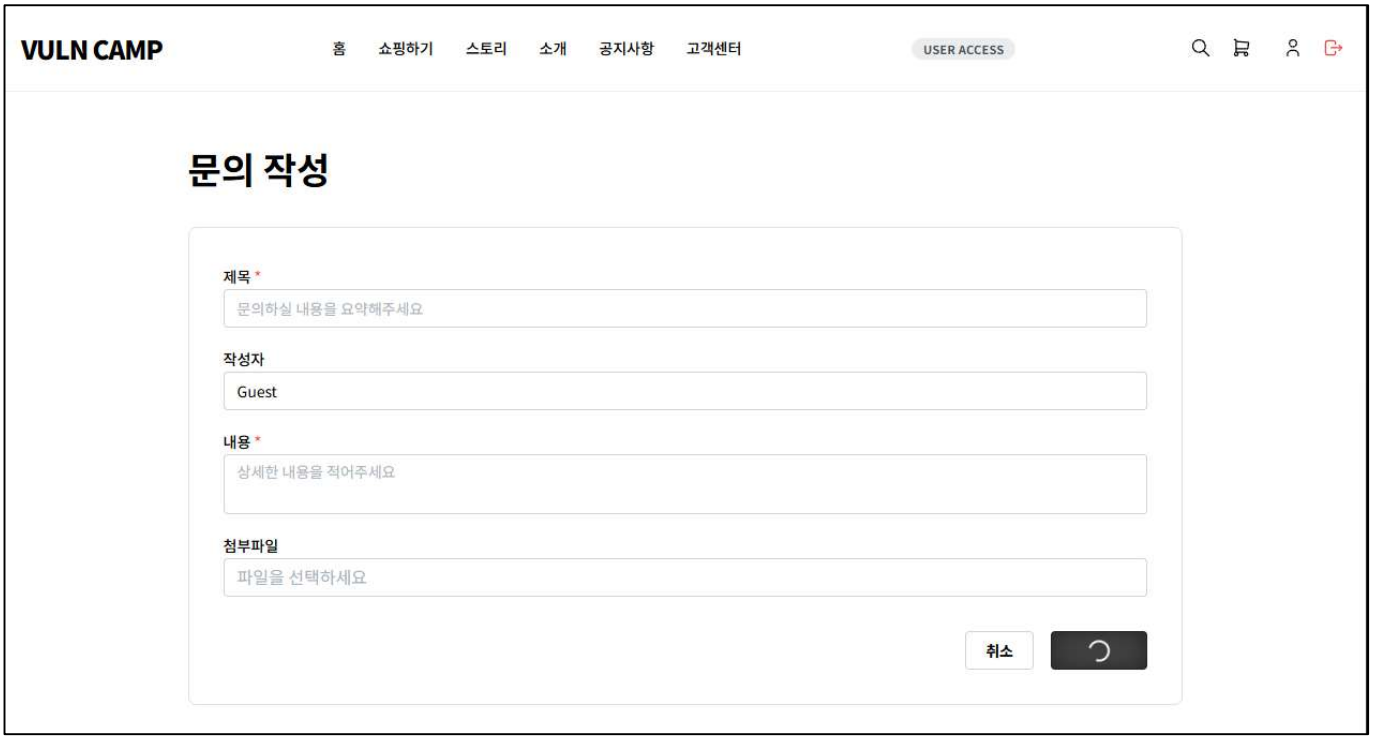
[그림 27] 쿠키 변조 후 공지사항 등록 결과 확인

4.1.7. XSS

Vuln-Camp 웹 애플리케이션의 QnA 게시글 작성 기능에서 사용자 입력값에 대한 검증이 미흡한 것을 확인하였음. 특히 게시글 본문에 입력된 스크립트 구문이 서버에 저장된 후 게시글 상세 조회 시 브라우저에서 실행될 수 있으며, 이를 통해 공격자가 사용자 세션 탈취, 악성 페이지 이동, 임의 스크립트 실행 등의 공격을 수행할 수 있음

.

QnA 게시글 작성 기능에서 제목, 작성자, 내용 값을 입력하여 게시글을 등록할 수 있는 기능이 존재하는 것을 확인하였음.



[그림 28] QnA 게시글 작성 기능 확인

QnA 게시글 작성 요청에서 title, content, author 파라미터에 XSS 구문을 삽입하여 전송한 결과, 서버 응답값에 입력한 스크립트 구문이 저장되어 반환되는 것을 확인하였음.

Request

Pretty Raw Hex

```
7  Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryDBoYFPxCgbA6dACQ
8  sec-ch-ua-mobile: ?0
9  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
10 Accept: */*
11 Origin: http://127.0.0.1:3001
12 Sec-Fetch-Site: cross-site
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Referer: http://127.0.0.1:3001/
16 Accept-Encoding: gzip, deflate, br
17 Connection: keep-alive
18
19 ------WebKitFormBoundaryDBoYFPxCgbA6dACQ
20 Content-Disposition: form-data; name="title"
21
22 test<>'"()
23 ------WebKitFormBoundaryDBoYFPxCgbA6dACQ
24 Content-Disposition: form-data; name="content"
25
26 test<>'"()
27 ------WebKitFormBoundaryDBoYFPxCgbA6dACQ
28 Content-Disposition: form-data; name="author"
29
30 test<>'"()
31 ------WebKitFormBoundaryDBoYFPxCgbA6dACQ--
32
```

Search 0 highlights

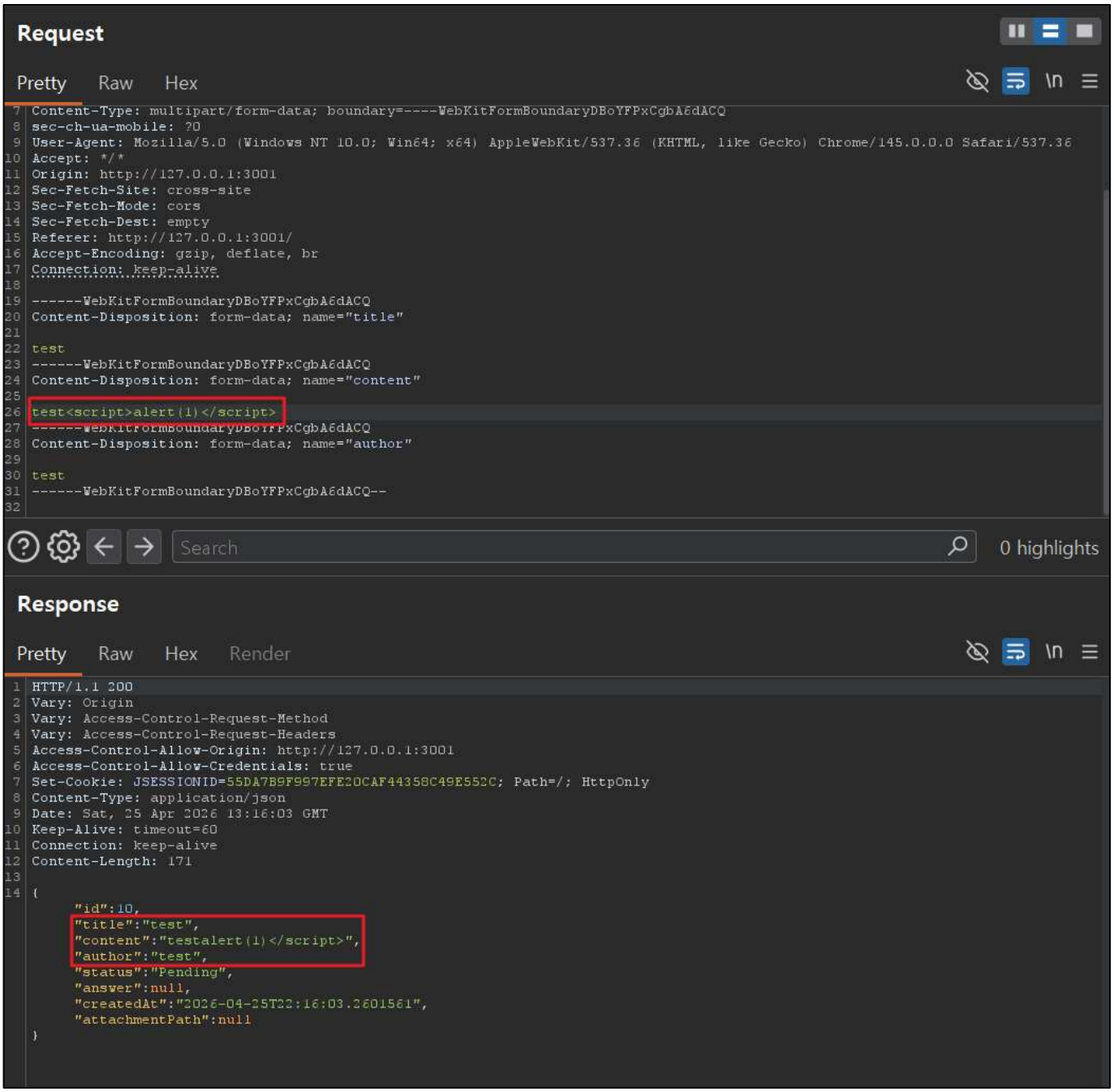
Response

Pretty Raw Hex Render

```
1  HTTP/1.1 200
2  Vary: Origin
3  Vary: Access-Control-Request-Method
4  Vary: Access-Control-Request-Headers
5  Access-Control-Allow-Origin: http://127.0.0.1:3001
6  Access-Control-Allow-Credentials: true
7  Set-Cookie: JSESSIONID=4EF2D7032F491132D7F70BFC7FD941EF; Path=/; HttpOnly
8  Content-Type: application/json
9  Date: Sat, 25 Apr 2026 13:14:24 GMT
10 Keep-Alive: timeout=60
11 Connection: keep-alive
12 Content-Length: 160
13
14 {
       "id":9,
       "title":"test",
       "content":"test<>'\"()",
       "author":"test",
       "status":"Pending",
       "answer":null,
       "createdAt":"2026-04-25T22:14:24.2343001",
       "attachmentPath":null
   }
```

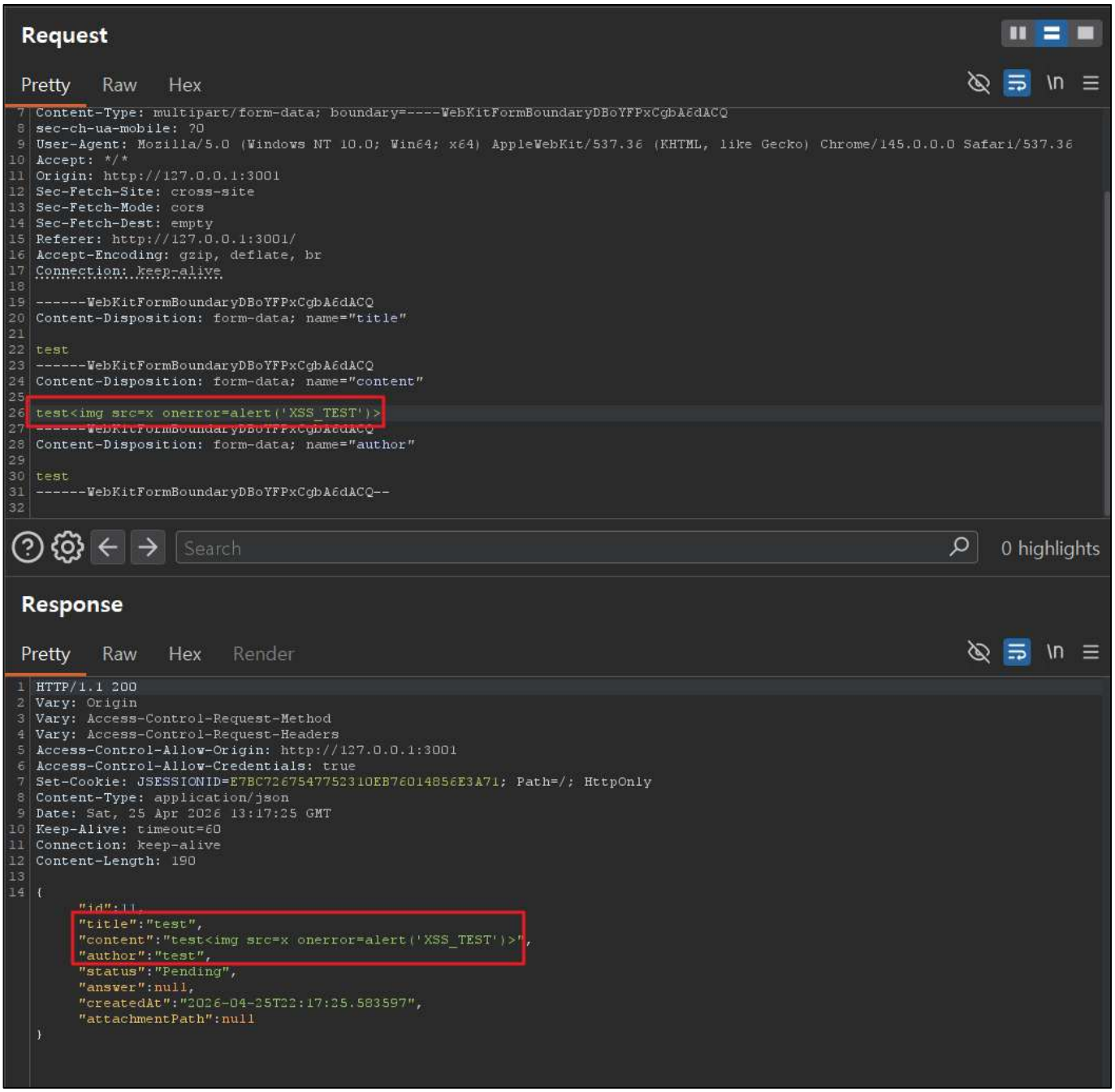
[그림 29] XSS 구문 삽입 및 저장 확인

QnA 게시글 작성 요청의 content 파라미터에 <script> 태그 기반 XSS 구문을 삽입하여 전송한 결과, 서버 응답값에서 <script> 시작 태그만 제거되고 스크립트 구문의 일부가 그대로 반환되는 것을 확인하였음.



[그림 30] <script> 구문 필터링 확인

QnA 게시글 작성 요청의 content 파라미터에 이미지 태그의 onerror 이벤트 핸들러를 이용한 XSS 우회 구문을 삽입하여 전송한 결과, 서버 응답값에 해당 구문이 필터링되지 않고 그대로 저장되는 것을 확인하였음.



[그림 31] 이벤트 핸들러 기반 XSS 구문 저장 확인

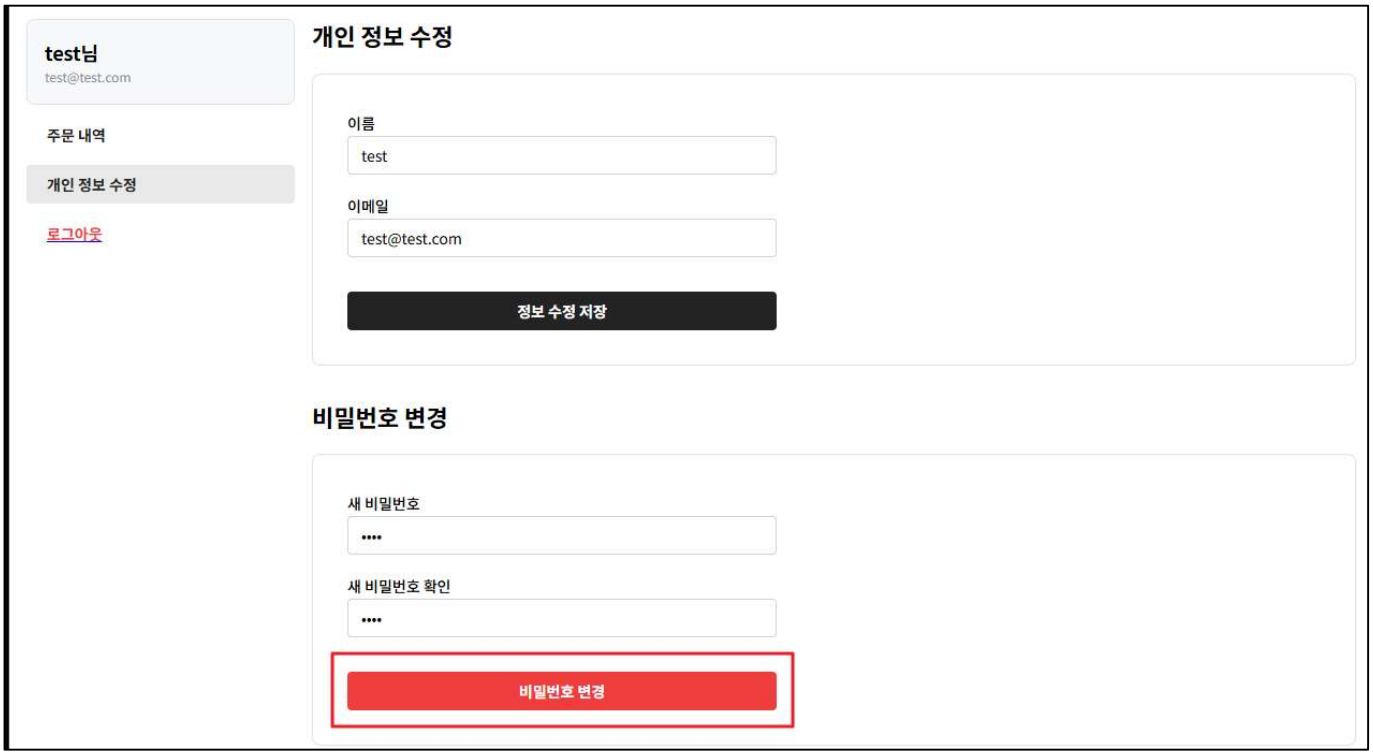
저장된 QnA 게시글 상세 페이지에 접근한 결과, 본문에 삽입된 onerror 이벤트 기반 XSS 구문이 브라우저에서 실행되어 alert 창이 출력되는 것을 확인하였음.

[그림 32] Stored XSS 실행 확인

4.1.8. CSRF

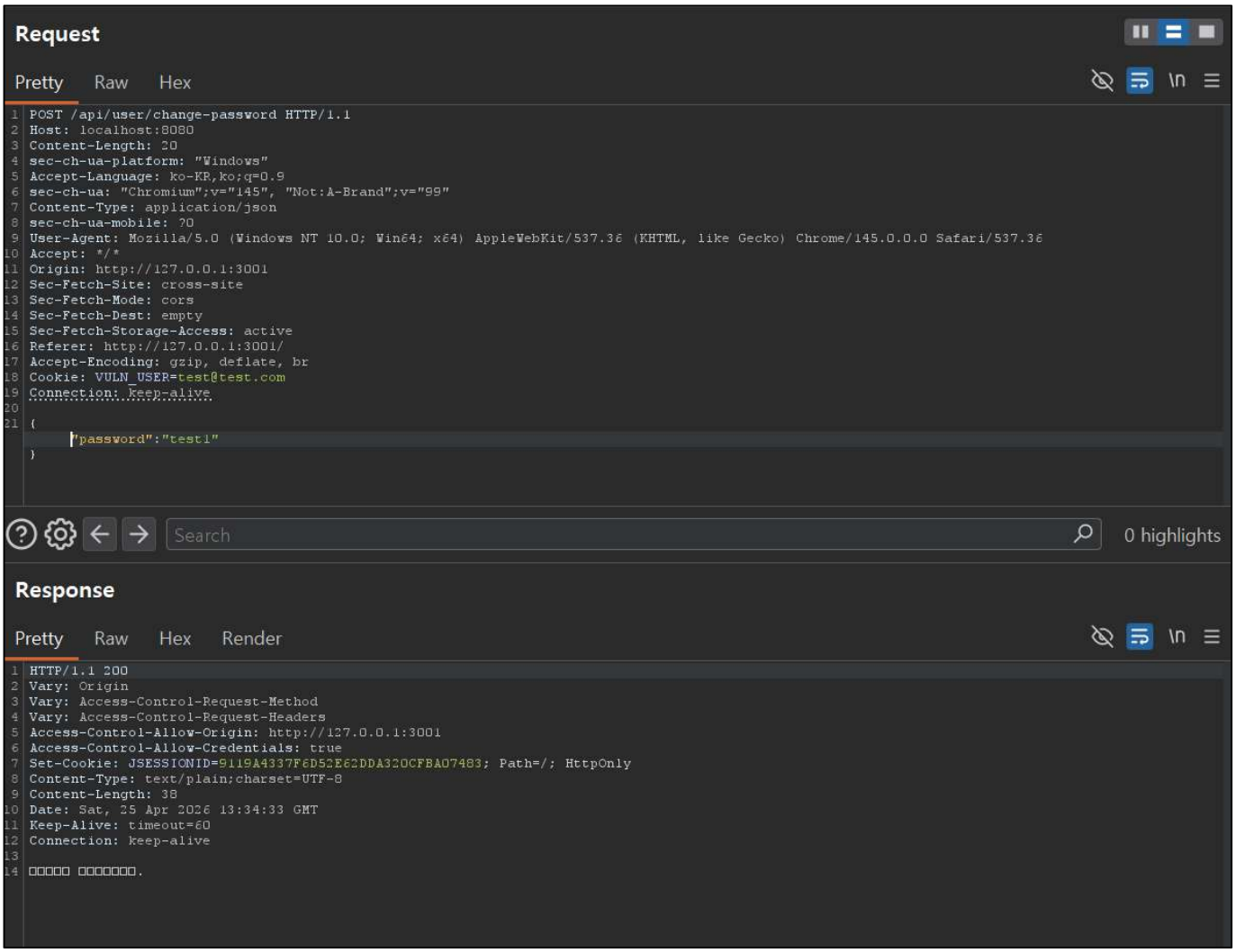
Vuln-Camp 웹 애플리케이션의 비밀번호 변경 기능에서 CSRF 토큰 검증 및 기존 비밀번호 확인 절차가 미흡한 것을 확인하였음. 해당 기능은 사용자 인증 세션만 존재하면 요청을 처리하므로, 공격자가 조작한 외부 페이지를 통해 피해자의 의도와 무관하게 비밀번호 변경 요청을 전송할 수 있음.

마이페이지의 비밀번호 변경 기능에서 새 비밀번호와 확인 값만 입력하면 비밀번호 변경 요청을 수행할 수 있는 것을 확인하였음.



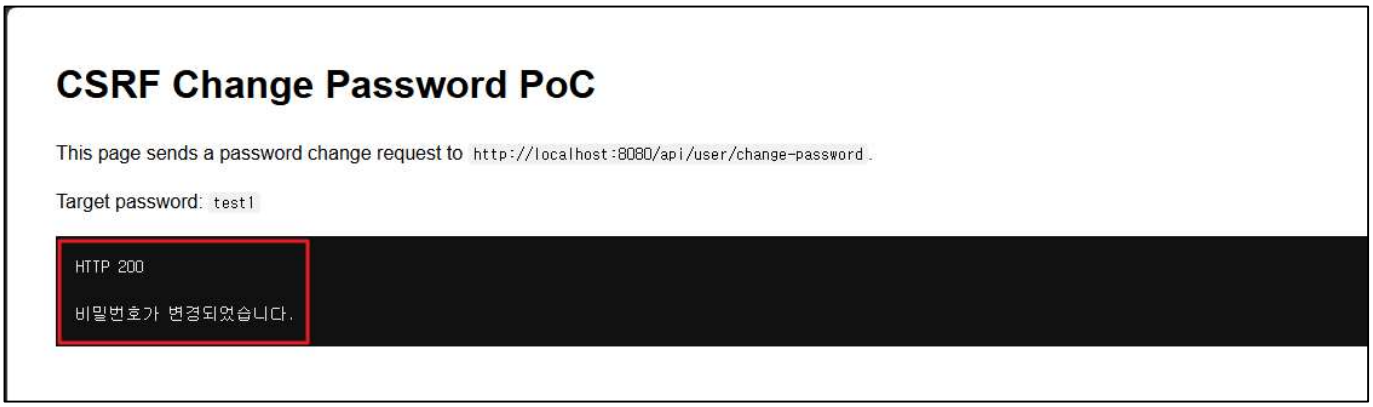
[그림 33] 비밀번호 변경 기능 확인

외부 CSRF PoC 페이지에서 비밀번호 변경 요청을 자동 전송한 결과, VULN_USER 쿠키가 포함된 상태에서 요청이 처리되어 비밀번호가 변경되는 것을 확인하였음.



[그림 34] CSRF를 통한 비밀번호 변경 요청 성공 확인

CSRF PoC 페이지 접근 시 /api/user/change-password 요청이 자동으로 전송되었으며, CSRF 토큰 및 기존 비밀번호 검증 없이 HTTP 200 응답과 함께 비밀번호 변경이 처리되는 것을 확인하였음.

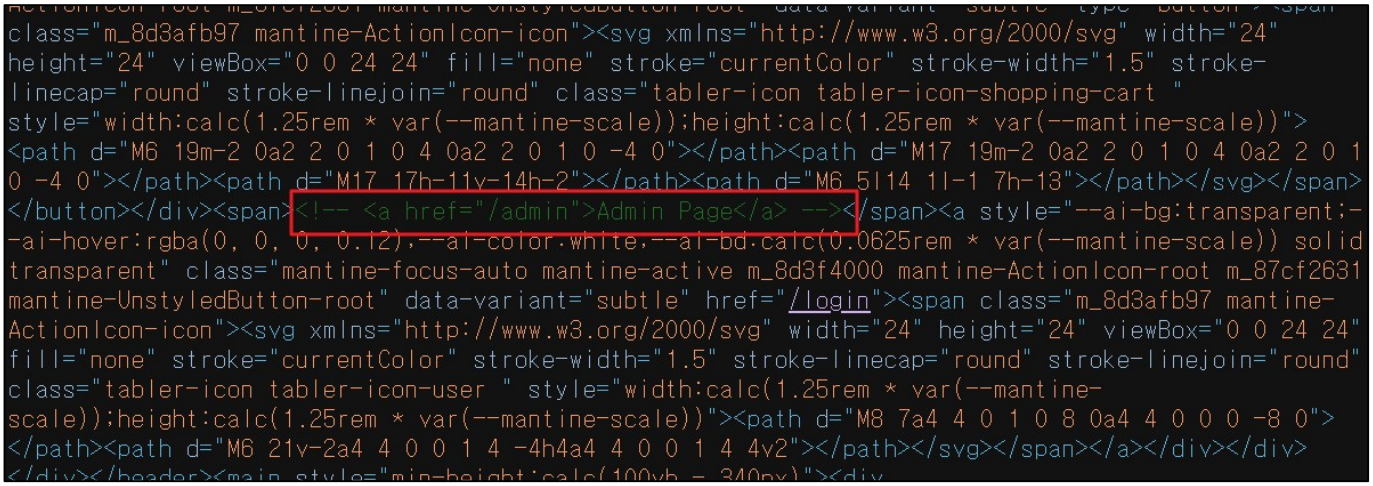


[그림 35] CSRF 공격 페이지 실행 결과 확인

4.1.9. 정보 누출

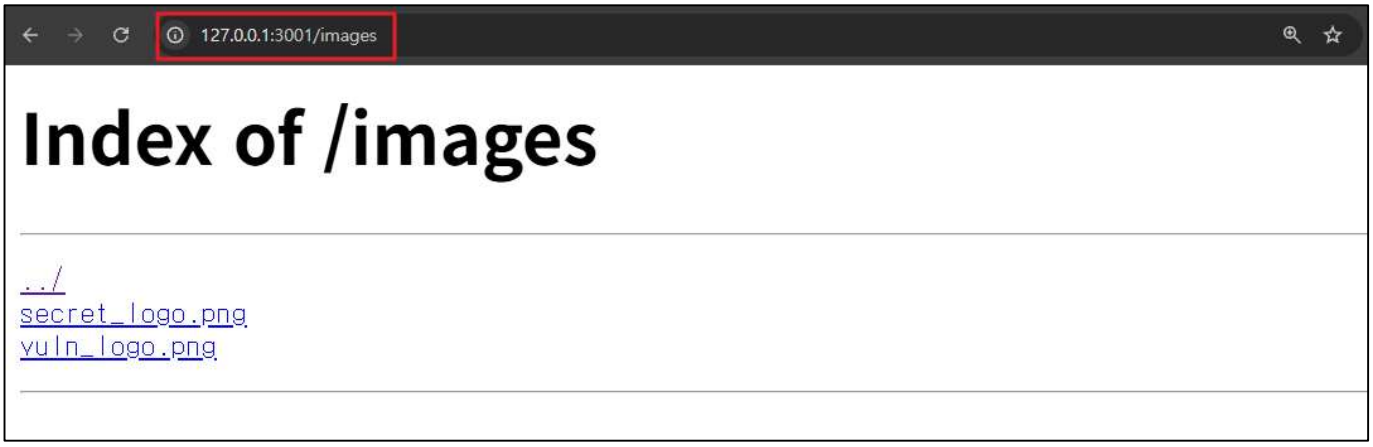
Vuln-Camp 웹 애플리케이션의 프론트엔드 소스 및 정적 리소스 경로에서 불필요한 정보가 노출되는 것을 확인하였음. 페이지 소스 내 개발자 주석을 통해 관리자 페이지 경로가 노출되었으며, /images 경로 접근 시 디렉토리 내 파일 목록이 인덱싱되어 표시되는 것을 확인하였음. 이를 통해 공격자가 관리자 기능 경로 및 서버 내 리소스 구조를 파악하여 후속 공격에 활용할 수 있음.

페이지 소스 보기 기능을 통해 HTML 소스 내 개발자 주석을 확인한 결과, 관리자 페이지 경로(/admin)가 노출되는 것을 확인하였음.



[그림 36] HTML 주석 내 관리자 페이지 경로 노출 확인

/images 경로에 직접 접근한 결과, 서버 내 이미지 디렉토리의 파일 목록이 인덱싱되어 노출되는 것을 확인하였음.



[그림 37] 디렉토리 리스팅을 통한 파일 목록 노출 확인

4.1.10. 서버 정보 노출

Vuln-Camp 웹 애플리케이션의 디버그 API에서 서버 내부 소스코드 및 구현 정보가 노출되는 것을 확인하였음. 특히 /api/debug/source 기능을 통해 서버 측 파일 경로를 지정할 경우 내부 소스코드가 응답으로 반환되며, 이를 통해 공격자가 API 구조, 컨트롤러 경로, 내부 로직 등 서버 구현 정보를 확인할 수 있음.

/api/debug/source 기능에 서버 내부 파일 경로를 입력한 결과, 서버 측 소스코드가 응답으로 반환되는 것을 확인하였음.

localhost:3001/api/debug/source?file=src/main/java/com/vulnlab/web/controller/AuthController.java

```
package com.vulnlab.web.controller;

import com.vulnlab.web.model.User;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;

import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/api")
public class AuthController {

    @PersistenceContext
    private EntityManager entityManager;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody Map<String, String> credentials) {
        String username = credentials.get("username");
        String password = credentials.get("password");

        User user = userRepository.findByUsername(username);

        if (user != null) {

            if (user.getPassword().equals(password)) {

                // 原◆由ㅂ역 怨쑣절 ◆뺤씔 (Admin 沅쑷뷜 遣◆◆◆)
                String role = "user";
                if ("admin@vuln.camp".equals(user.getUsername())) {
                    role = "admin";
                }

                return ResponseEntity.ok()
                        .header("Set-Cookie", "VULN_ROLE=" + role + "; Path=/")
                        .header("Set-Cookie", "VULN_USER=" + user.getUsername() + "; Path=/")
                        .body(java.util.Collections.singletonMap("message", "Login successful"));
            } else {
                return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid password");
            }
        } else {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("User not found");
        }
    }

    // ◆쑣쑬 濡씟렇◆◆ ◆◆뷜 謙고쉶
    @GetMapping("/role")
    public ResponseEntity<?> checkRole(@CookieValue(value = "VULN_ROLE", defaultValue = "guest") String role) {
        Map<String, String> response = new HashMap<>();
        response.put("role", role);
        if ("admin".equals(role)) {
            response.put("message", "Welcome, Administrator!");
            response.put("flag", "FLAG{COOKIE_TAMPERING_SUCCESS}");
        } else {
            response.put("message", "Welcome, User.");
        }
        return ResponseEntity.ok(response);
    }

    @Autowired
    private com.vulnlab.web.repository.UserRepository userRepository;

    @PostMapping("/signup")
    public ResponseEntity<?> signup(@RequestBody User user) {
        try {
            if (userRepository.findByUsername(user.getUsername()) != null) {
                return ResponseEntity.badRequest().body("◆대◆ 謙댁쑬◆샔뒝 ◆ㅂ 슛◆멧였◆댐땜.");
            }
            userRepository.save(user);
            return ResponseEntity.ok("◆쑷쌥嫄◆◆낆씄 ◆쑰즺◆샣뽐◆듬똼◆◆.");
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("◆쑷쌥嫄◆◆◆ 以◆ ◆ㅅ쩽嫄◆ 諛쒓깽◆댐뒹◆댐땜.");
        }
    }
}
```

[그림 37] 디버그 API를 통한 서버 소스코드 노출 확인

5. 웹 취약점 보안대책

5.1. SQL 인젝션

취약점 개요	
설명	SQL 인젝션은 사용자 입력값이 SQL 질의문에 안전하게 처리되지 않아 공격자가 임의의 SQL 구문을 삽입할 수 있는 취약점임. 본 진단에서는 QnA 게시판의 날짜 검색 기능에서 startDate, endDate 파라미터 값이 SQL 조건절에 직접 사용되어 검색 조건 우회가 가능한 것을 확인하였음.
보안위협	공격자는 조작된 SQL 구문을 삽입하여 정상 검색 조건을 우회하거나 데이터베이스 질의를 변조할 수 있음. 이를 통해 게시글 정보 등 데이터베이스에 저장된 정보가 비정상적으로 조회될 수 있으며, 공격 구문에 따라 중요 정보 유출 및 데이터 변조로 이어질 수 있음.
조치방법	
사용자 입력값을 SQL 질의문에 직접 연결하지 않고 PreparedStatement 또는 ORM 파라미터 바인딩을 적용해야 함. 날짜 파라미터는 서버 측에서 YYYY-MM-DD 형식으로 검증하고, 허용되지 않은 문자 및 SQL 예약어가 포함된 요청은 차단해야 함. 또한 데이터베이스 계정 권한을 최소화하고, SQL 오류 메시지가 사용자에게 노출되지 않도록 예외 처리를 적용해야 함.	

5.2. SSRF

취약점 개요	
설명	SSRF는 사용자가 입력한 URL을 서버가 대신 요청하는 과정에서 내부망 또는 로컬 자원에 접근할 수 있는 취약점임. 본 진단에서는 /api/ssrf/fetch API의 url 파라미터를 통해 서버 측 요청이 수행되는 것을 확인하였음.
보안위협	공격자는 내부망 주소 또는 로컬 주소를 입력하여 외부에서 직접 접근할 수 없는 내부 서비스의 응답 정보를 확인할 수 있음. 이를 통해 내부 API 정보, 서버 구조, 민감한 응답 데이터가 노출될 수 있음.
조치방법	
서버 측 요청 기능에는 허용된 도메인만 요청할 수 있도록 화이트리스트를 적용해야 함. localhost, 127.0.0.1, 사설 IP 대역, 메타데이터 주소 등 내부망 접근은 차단하고, 리다이렉트 이후 최종 URL도 검증해야 함.	

5.3. 파일 업로드

취약점 개요	
설명	파일 업로드 취약점은 업로드 파일의 확장자, MIME 타입, 파일 내용 검증이 미흡하여 악성 파일이 서버에 저장되거나 실행될 수 있는 취약점임. 본 진단에서는 QnA 첨부파일 기능을 통해 JSP 파일 업로드 및 실행이 가능한 것을 확인하였음.
보안위협	공격자가 서버 측에서 실행 가능한 파일을 업로드할 경우 웹쉘 실행, 서버 명령 수행, 내부 파일 접근 등으로 이어질 수 있음. 이는 서버 장악 및 추가 침해의 원인이 될 수 있음.
조치방법	
업로드 가능한 확장자를 화이트리스트 방식으로 제한하고, 파일 내용과 MIME 타입을 함께 검증해야 함. 업로드 파일은 웹 루트 외부에 저장하고 실행 권한을 제거해야 하며, 저장 파일명은 난수화하여 직접 접근을 차단해야 함.	

5.4. 파일 다운로드

취약점 개요	
설명	파일 다운로드 취약점은 다운로드 대상 파일명을 사용자가 조작할 수 있고 경로 검증이 미흡할 때 발생하는 취약점임. 본 진단에서는 filename 파라미터에 상위 디렉터리 이동 구문을 삽입하여 서버 내부 설정 파일 다운로드가 가능한 것을 확인하였음.
보안위협	공격자는 서버 내부 설정 파일, 소스코드, 로그 파일 등 비공개 파일을 다운로드할 수 있음. 이를 통해 DB 설정, 서버 포트, 내부 경로 등 후속 공격에 활용 가능한 정보가 노출될 수 있음.
조치방법	
사용자가 전달한 파일명을 직접 경로에 사용하지 말고, 서버에 저장된 파일 식별자와 매핑하여 다운로드해야 함. ../ 등 경로 이동 문자를 차단하고, 다운로드 허용 디렉터리 밖의 파일 접근을 제한해야 함.	

5.5. 프로세스 검증 누락

취약점 개요	
설명	프로세스 검증 누락은 업무 처리 과정에서 서버 측 검증이 충분히 수행되지 않아 클라이언트 요청값 조작만으로 비정상 처리가 가능한 취약점임. 본 진단에서는 결제 금액 파라미터 변조 및 쿠폰 검증 로직 노출을 확인하였음.
보안위협	공격자는 결제 금액을 임의로 조작하거나 쿠폰 생성 규칙을 분석하여 할인 혜택을 비정상적으로 적용할 수 있음. 이로 인해 금전적 피해 및 서비스 정책 우회가 발생할 수 있음.
조치방법	
결제 금액, 할인율, 쿠폰 유효성 등 핵심 업무 값은 서버 측에서 신뢰 가능한 데이터 기준으로 재계산해야 함. 클라이언트에서 전달된 금액이나 검증 결과를 신뢰하지 말고, 쿠폰 발급 여부와 사용 권한을 서버에서 검증해야 함.	

5.6. 불충분한 인가

취약점 개요	
설명	불충분한 인가는 사용자 권한 검증이 미흡하여 권한이 없는 사용자가 관리자 기능 또는 제한된 기능을 수행할 수 있는 취약점임. 본 진단에서는 VULN_ROLE 쿠키 값을 admin으로 변조하여 공지사항 작성 기능에 접근 가능한 것을 확인하였음.
보안위협	공격자는 클라이언트 쿠키 값을 조작하여 관리자 권한으로 인식될 수 있으며, 이를 통해 공지사항 작성 등 관리자 기능을 비인가 상태로 수행할 수 있음.
조치방법	
권한 정보는 클라이언트 쿠키 값만으로 판단하지 말고 서버 세션 또는 서명된 토큰을 기반으로 검증해야 함. 관리자 기능 요청 시 매 요청마다 서버 측 권한 검사를 수행하고, 쿠키에는 민감한 권한 정보를 평문으로 저장하지 않아야 함.	

5.7. XSS

취약점 개요	
설명	XSS는 사용자 입력값이 웹 페이지에 출력될 때 안전하게 인코딩되지 않아 브라우저에서 스크립트가 실행되는 취약점임. 본 진단에서는 QnA 게시글 본문에 이벤트 핸들러 기반 XSS 구문이 저장되고 상세 조회 시 실행되는 것을 확인하였음.
보안위협	공격자는 악성 스크립트를 삽입하여 사용자 쿠키 탈취, 악성 페이지 이동, 임의 요청 수행 등을 유도할 수 있음. 저장형 XSS의 경우 게시글을 조회하는 다수 사용자에게 지속적으로 영향을 줄 수 있음.
조치방법	
사용자 입력값 출력 시 HTML 컨텍스트에 맞는 출력 인코딩을 적용해야 함. script 태그뿐 아니라 이벤트 핸들러, img 태그, SVG 등 우회 구문도 필터링해야 하며, 허용 태그 기반의 HTML Sanitizer와 CSP 적용을 권장함.	

5.8. CSRF

취약점 개요	
설명	CSRF는 사용자가 로그인된 상태에서 공격자가 유도한 요청이 사용자의 의도와 무관하게 서버로 전송되는 취약점임. 본 진단에서는 비밀번호 변경 기능에서 CSRF 토큰 및 기존 비밀번호 검증 없이 요청이 처리되는 것을 확인하였음.
보안위협	공격자는 피해자가 로그인된 브라우저를 이용하여 비밀번호 변경 등 중요 기능 요청을 강제로 수행시킬 수 있음. 이로 인해 계정 탈취 또는 사용자 설정 변경이 발생할 수 있음.
조치방법	
상태 변경 요청에는 CSRF 토큰을 적용하고 서버에서 토큰 유효성을 검증해야 함. SameSite 쿠키 속성을 Strict 또는 Lax로 설정하고, 비밀번호 변경 등 중요 기능에는 기존 비밀번호 확인 또는 재인증 절차를 적용해야 함.	

5.9. 정보 누출

취약점 개요	
설명	정보 누출은 웹 페이지, HTML 주석, 디렉터리 리스팅, API 응답 등을 통해 불필요한 정보가 사용자에게 노출되는 취약점임. 본 진단에서는 HTML 주석 및 디렉터리 리스팅을 통해 관리자 경로와 서버 파일 목록이 노출되는 것을 확인하였음.
보안위협	공격자는 노출된 관리자 경로, 파일 목록, 내부 구조 정보를 이용하여 추가 공격 대상을 식별할 수 있음. 이는 인증 우회, 파일 다운로드, 소스코드 노출 등 후속 공격에 활용될 수 있음.
조치방법	
운영 환경에서는 HTML 주석, 디버그 메시지, 불필요한 경로 정보가 노출되지 않도록 제거해야 함. 디렉터리 리스팅 기능은 비활성화하고, 정적 파일 접근은 필요한 파일에 한해 제한적으로 허용해야 함.	

5.10. 서버 정보 노출

취약점 개요	
설명	서버 정보 노출은 디버그 기능이나 오류 응답 등을 통해 서버 내부 소스코드, 경로, 구현 정보가 노출되는 취약점임. 본 진단에서는 /api/debug/source 기능을 통해 서버 내부 소스코드가 반환되는 것을 확인하였음.
보안위협	공격자는 노출된 소스코드를 통해 API 경로, 취약한 로직, 내부 파일 구조를 파악할 수 있음. 이는 SSRF, 권한 우회, 파일 다운로드 등 다른 취약점 공격을 고도화하는 데 활용될 수 있음.
조치방법	
운영 환경에서는 디버그 API와 소스코드 조회 기능을 제거해야 함. 개발용 기능은 인증된 관리자에게만 제한하거나 별도 내부망에서만 접근 가능하도록 구성하고, 소스 파일이 웹 응답으로 반환되지 않도록 차단해야 함.	